

Evaluating the Use of LLMs for Documentation to Code Traceability

EBUBE ALOR, Concordia University, Canada

SAYEDHASSAN KHATOONABADI, Concordia University, Canada

EMAD SHIHAB, Concordia University, Canada

Large Language Models (LLMs) offer new potential for automating documentation-to-code traceability, yet their capabilities and limitations remain underexplored. We evaluate three LLMs (Claude Sonnet, GPT-4o, and o3-mini) on documentation-to-code traceability across diverse software projects. We create two datasets from open-source systems (Unity Catalog and Crawl4AI), curated to minimize training-data overlap with the evaluated models, and derive a 27-project benchmark from CoDocBench and TransArc to study generalizability across established codebases. Our experiments assess three capabilities: (1) trace link identification, (2) relationship explanation quality, and (3) multi-step chain reconstruction, while comparing LLMs against TF-IDF, BM25, and CodeBERT baselines. On the curated datasets, the best LLM attains F1-scores of 79.4% and 80.4%, and on the Established Projects benchmark, LLMs reach average F1 up to 92.3%, substantially outperforming traditional baselines except in a single dense system. Fully correct explanations cover 42.9%–71.1% of links, while partial correctness exceeds 97%, and chain endpoints are almost always recovered, though intermediate steps are often imperfect. We also show that task framing, particularly one-to-many matching, is crucial for recall, whereas naive many-to-many prompts achieve around 5% F1 due to enumeration bias. Overall, LLMs are promising assistants for trace discovery when combined with human oversight and careful tool design.

CCS Concepts: • **Software and its engineering** → **Maintaining software**; **Software evolution**; • **Computing methodologies** → **Natural language processing**.

Additional Key Words and Phrases: Software traceability, documentation traceability, code traceability, software artifacts

ACM Reference Format:

Ebube Alor, SayedHassan Khatoonabadi, and Emad Shihab. 2026. Evaluating the Use of LLMs for Documentation to Code Traceability. 1, 1 (June 2026), 54 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Traceability in software engineering refers to the ability to establish and maintain relationships between various software artifacts produced throughout the software development lifecycle [22, 23]. These artifacts include requirements, design documents, source code, test cases, and documentation [53, 74]. Effective traceability enables stakeholders to understand how different artifacts

Authors' Contact Information: Ebube Alor, Data-driven Analysis of Software (DAS) Lab, Department of Computer Science & Software Engineering, Concordia University, Montréal, QC, Canada, ebubechukwu.alor@mail.concordia.ca; SayedHassan Khatoonabadi, Data-driven Analysis of Software (DAS) Lab, Department of Computer Science & Software Engineering, Concordia University, Montréal, QC, Canada, sayedhassan.khatoonabadi@mail.concordia.ca; Emad Shihab, Data-driven Analysis of Software (DAS) Lab, Department of Computer Science & Software Engineering, Concordia University, Montréal, QC, Canada, emad.shihab@concordia.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/6-ART

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

influence one another, facilitating tasks such as impact analysis, change management, and compliance verification [23]. Documentation-to-code traceability focuses on linking natural language documentation to the corresponding code implementations [9, 18]. Documentation plays a vital role in software development by aiding in communication among team members, providing guidance for software usage, and serving as a reference for maintenance and future development [73]. It helps developers and users understand the system’s functionality, design decisions, and usage instructions, which is essential for the success and longevity of software projects.

However, establishing and maintaining accurate traceability between documentation and code presents significant challenges. While the connections between documentation and code may be clear to the original author during creation, they often become obscure as the software evolves. Code refactoring, renaming, and architectural changes lead to “documentation decay” where the documentation persists but the underlying code links are broken [3, 85]. Furthermore, for new team members who lack the original author’s mental model, navigating from developer-facing documentation, such as API references, tutorials, architectural descriptions, and function-level documentation, to the corresponding code implementations can be a non-trivial task, especially when the relevant code is scattered across multiple elements. Automating this traceability is difficult because documentation is often written in free-flowing, natural language that is unstructured and may contain ambiguities [18]. Traditional traceability methods, such as Information Retrieval (IR) techniques, rely on keyword matching and statistical analysis, which may not capture the deeper semantic relationships between documentation and code [24, 52]. These methods suffer from low precision or recall, leading to incomplete or incorrect traceability links [24, 52]. As software systems evolve and codebases grow in size and complexity, scalability becomes a critical concern [37, 100]. Existing methods may require substantial manual effort to maintain traceability links and may not adapt well to changes in the code or documentation [79, 88]. Additionally, they may be limited to specific programming languages or domains, hindering their applicability in diverse settings [37].

Recent advancements in Large Language Models (LLMs) offer promising opportunities to address these challenges [29, 44, 63, 97]. LLMs, such as OpenAI’s GPT series, have demonstrated exceptional capabilities in understanding and generating human-like text, as well as in processing and generating code [17, 29, 63, 70]. Their ability to comprehend natural language semantics and code syntax suggests the potential for improving documentation-to-code traceability by bridging the gap between unstructured documentation and structured code artifacts. Despite the potential of LLMs, there is a lack of comprehensive studies evaluating their effectiveness in documentation-to-code traceability.

To address this knowledge gap, we conduct a comprehensive evaluation of LLMs for documentation-to-code traceability tasks. We construct datasets from 29 open-source projects: two “unseen” projects developed after the LLMs’ training cutoff date (April 2024) to control for data contamination, and 27 established projects comprising 25 Python libraries derived from CoDocBench [71] and two Java microservice systems (MediaStore and TeaStore) from the TransArc benchmark [33]. These projects feature diverse developer-documentation styles, including API references, tutorial-style documentation, function-level docstrings, and sentence-level architectural descriptions, along with their corresponding code artifacts at different granularity levels (classes, methods, and statements). Importantly, these datasets play complementary roles in our evaluation: the unseen projects and TransArc cover richer API/tutorial- and architecture-facing documentation, while CoDocBench provides a scalable benchmark for function-level developer documentation across many mature projects.

Our methodology employs a systematic evaluation approach where we assess three leading LLMs (Claude Sonnet 3.5, GPT-4o, and o3-mini) against established baselines (TF-IDF, BM25, and CodeBERT) using a one-to-many matching strategy—where each documentation segment is

evaluated against all potential code artifacts simultaneously rather than examining each possible pair individually. For each LLM, we examine its ability to establish trace links, explain relationships, and reconstruct dependency chains between documentation and code. Based on this, our study aims to investigate the following three research questions:

RQ1: *How accurately can LLMs identify trace links between documentation segments and their corresponding code elements?* While LLMs demonstrate code-text understanding, their fundamental capability to map documentation to precise code elements remains unproven. Traditional IR methods struggle with the documentation's hybrid structure (mix of formal specs and natural language) and implicit relationships. Establishing this baseline capability is critical before exploring advanced traceability applications. We define TF-IDF, BM25, and CodeBERT as our baselines. On our manually constructed unseen datasets (Crawl4AI and Unity Catalog), the best-performing LLM achieves F1-scores of 79.4% and 80.4%, respectively, substantially outperforming the best baseline scores of 54.2% and 69.3%. On the Established Projects benchmark (25 CoDocBench libraries and 2 TransArc systems), LLMs again dominate: o3-mini attains an average F1-score of 92.3% across 27 projects, compared to 60.5% for the strongest baseline (TF-IDF).

RQ2: *How effectively can LLMs explain the nature of relationships between documentation and code elements?* Traceability can require more than binary links (is related vs not related). For example, maintenance tasks demand an understanding of how code implements documented functionality. While LLMs are capable of generating natural language explanations, their capacity to produce technically precise relationship descriptions (e.g., "implements validation rule" vs "configures service") remains unexplored. We evaluate whether LLMs advance beyond linkage to semantic understanding crucial for developer trust and actionable insights. We find that while fully correct relationship explanations range from 42.9% to 71.1%, partial accuracy exceeds 97%, indicating that fundamental connections are rarely missed.

RQ3: *How completely can LLMs identify intermediate elements in documentation-to-code trace chains?* Real-world traceability often involves multi-step connections (e.g., documentation → interface → class → method). Prior work often focuses on direct links (documentation → method), but software evolution requires understanding dependency chains. We evaluate whether LLMs can reconstruct these pathways—a capability essential for impact analysis and change management in complex systems. We find that for multi-step chains, LLMs maintain high endpoint accuracy (below 2% incorrect) but vary in capturing precise intermediate links from 13% to 80%.

While our results show that LLMs outperform traditional baselines in identifying trace links, they also reveal limitations, particularly in generating fully complete explanations and tracing multi-step dependencies. We then turn our analysis from what the LLMs achieve to how task-framing influences the performance. Specifically, we investigate the impact of context management strategies and additional contextual information on traceability performance. Based on our findings, we offer practical recommendations: for developers, LLMs can be valuable traceability assistants when paired with human oversight; and for tool designers, strategies like one-to-many linking and human-in-the-loop review can enhance usability. Finally, we outline directions for future LLM research to address current limitations in explanation depth and intermediate trace reconstruction.

Contributions. Our key contributions include:

- **Dataset Creation:** We created two novel, manually curated datasets from unseen projects (Unity Catalog and Crawl4AI), and we derived a 27-project Established Projects benchmark

by adapting CoDocBench [71] and TransArc [33] into a unified documentation-to-code traceability format.

- **Comprehensive Evaluation:** We provide empirical evidence on LLMs for documentation-to-code traceability tasks, examining their ability to classify and map documentation to code accurately.
- **Relationship Identification:** We explore the capability of LLMs to identify and describe the specific types of relationships between documentation and code, moving beyond simple linkage to semantic understanding.
- **Practical Implications:** We discuss recommendations for integrating LLMs into software development practices to enhance traceability and support maintenance activities.

Paper Organization. The remainder of this paper is organized as follows. Section 2 provides background information and reviews related work on documentation-to-code traceability and the application of LLMs in software engineering. Section 3 describes the methodology used for our evaluation, including dataset preparation and experimental design. Sections 4, 5 and 6 presents the experimental results and analysis for each research question. Section 7 discusses additional analysis in our paper. Section 8 discusses the implications of our findings for practitioners, tool designers, and researchers. Section 9 highlights threats to the validity of the study. Finally, Section 10 concludes the paper.

2 BACKGROUND AND RELATED WORK

In this section, we provide an overview of the key concepts and terminology essential for understanding our work. First, we discuss software documentation and its complexities. Next, we look into software traceability, focusing specifically on the critical challenge of linking documentation to code. We then review traditional automated approaches for traceability and their limitations. Finally, we discuss the potential of LLMs in software engineering, particularly their promise for addressing documentation-to-code traceability challenges.

2.1 Software Documentation

Software documentation serves as the primary medium for communicating software system knowledge between developers, maintainers, and users [26, 32]. While source code defines system behavior, documentation explains the intentions, usage patterns, and architectural decisions that shape a software system [73]. This critical role makes documentation essential for both software use and maintenance, and a key target for traceability efforts.

Modern software documentation comprises multiple forms, each addressing distinct objectives and audiences [2]. *User documentation* targets software users—for instance, end-users of applications or developers who are consumers of a library or service—by presenting tutorials, usage guides, and troubleshooting steps. *Technical documentation* details architectural decisions and design rationale [26]. At a more code-centric level, *API documentation* often incorporates both automatically generated references (e.g., function signatures, endpoints) and explanatory text contextualizing system components [54]. These forms illustrate documentation’s range from high-level guidance to deep technical specifics [54].

However, maintaining documentation quality presents significant hurdles. A primary challenge is *consistency*: documentation often becomes out-of-sync with evolving code, a phenomenon known as *documentation decay* [3, 83–85]. This risk is amplified in rapid development cycles [83]. While practices like *documentation-as-code* [14] and automated generation tools [16, 49, 61, 99] help maintain structural references, they often fail to capture deeper *semantic* linkages [3]. Furthermore, documentation involves *varying levels of abstraction*, from high-level concepts to

implementation notes [27, 54], and often relies on *implicit* knowledge (like design rationale or constraints) not directly reflected in the code [83, 89]. These inherent complexities make establishing and maintaining clear connections to the codebase difficult [85, 89, 89].

In this work, we concentrate on three common and challenging documentation forms: user documentation (e.g., tutorials, guides), API documentation, and function-level documentation (e.g., docstrings) [19, 83]. These integrate natural language explanations with code references, providing an ideal testbed for evaluating LLM capabilities in linking textual and programmatic information [54]. Notably, these forms constitute *developer documentation* (artifacts intended to help developers understand and use code) as distinct from *requirements-level* artifacts such as architectural decision records (ADRs), compliance documents, or formal specifications, which represent a separate traceability problem addressed by prior work [5, 36, 39]. Our Established Projects dataset further includes short sentence-level architectural descriptions from the TransArc benchmark [33], capturing architecture-facing developer documentation that links high-level behavior to concrete implementation files.

2.2 Software Traceability and its Challenges

Traceability in software engineering refers to the ability to connect related software artifacts—such as requirements, designs, code, and documentation—throughout the lifecycle [22, 23]. It is crucial for tasks like impact analysis, change management, and compliance verification, enabling stakeholders to understand how changes propagate and ensure software aligns with its goals [23].

Historically, traceability links were maintained manually [22]. However, the scale of modern software systems, potentially involving millions of links, necessitates automation [83]. Beyond scale, software evolution poses a major traceability challenge: as features change and designs are updated, keeping artifacts synchronized becomes increasingly difficult [23, 85].

A particularly critical and challenging aspect is establishing reliable *documentation-to-code traceability* [9, 89]. This task is complicated by several factors inherent to the artifacts themselves:

- **Abstraction Gap:** Documentation often uses natural language to describe high-level concepts, rationale, or usage scenarios [52], while source code consists of formal, executable constructs [9, 56]. A single documentation paragraph might relate to multiple, scattered code components [45, 54, 89].
- **Evolution Mismatch:** Code is frequently refactored or renamed, while documentation updates may lag, leading to broken or outdated links [1, 2, 58, 83, 85].
- **Format and Language Differences:** Documentation and code use different languages and structures, making direct mapping complex [42].
- **Implicit Knowledge and Context:** Developers rely on implicit understanding of architecture or domain patterns to interpret documentation [46, 89]. Automated tools struggle to replicate this tacit knowledge, especially in large codebases or distributed systems (e.g., microservices) where context is fragmented [20, 38, 62, 98].

These factors significantly hinder the ability to establish and maintain accurate trace links between documentation and code using traditional methods [9, 89].

2.3 Automated Traceability Approaches

To address the challenges of manual tracing, various automated methods for trace link discovery have been developed. Early efforts relied on *information retrieval* (IR) techniques, such as keyword matching and text indexing [9]. While useful for smaller projects, basic IR struggles with varying terminology and lacks deep contextual understanding [42, 56].

Vector Space Models (VSM) [9, 52] and techniques like *Latent Semantic Indexing* (LSI) [42, 56] offered improvements by representing artifacts in vector spaces and capturing underlying semantic patterns, allowing identification of related concepts even with different terminology [41, 42, 58].

Subsequent *machine learning* (ML) approaches utilized supervised techniques, training models on manually verified links to identify similar patterns [21]. ML often augmented IR methods by incorporating deeper linguistic or domain-specific cues [55, 59, 60]. Many modern tools combine IR, ML, and sometimes rule-based systems to handle the multifaceted nature of trace relationships [18].

Natural language processing (NLP) has become integral to many approaches, especially for interpreting documentation [45]. However, these automated methods still face limitations. IR-based techniques may miss semantic connections expressed with divergent vocabulary. ML approaches typically require significant labeled training data and may struggle with project-specific conventions or context shifts. Rule-based systems require ongoing maintenance as software evolves [90]. The persistent gap between the capabilities of these traditional methods and the complexity of accurately tracing documentation to code, particularly in capturing nuances like design intent or usage constraints [54], drives the need for more robust, semantically enriched solutions [30, 91].

2.4 LLMs in Software Traceability

LLMs, such as OpenAI's GPT-4o and Anthropic's Claude Sonnet 3.5 have demonstrated remarkable capabilities in natural language understanding and generation [8, 47, 48]. A recent systematic literature review by Alturayef et al. [6] surveys the evolution from traditional IR techniques to machine-learning and deep-learning methods, including recent transformer-based approaches. Their review highlights that classical approaches struggle with the semantic gap between artifacts and motivates more semantically rich models, such as modern LLM-based techniques.

Recent empirical studies have begun exploring LLM capabilities for specific traceability tasks. Ge et al. [36] present the first approach for cross-level requirements tracing using LLMs, demonstrating that fine-tuned models with data augmentation strategies can effectively link requirements across abstraction levels. Other work applies LLMs to software traceability and reports superior performance over classical IR when linking requirements to code, mapping architecture documentation to source files, and tracing security requirements to goal models [5, 35, 39]. In the automotive domain, Hippargi et al. [43] evaluate LLM capabilities for traceability maintenance between system and software requirements, highlighting both the potential and current limitations in safety-critical contexts. Rodriguez et al. [78] explore prompt engineering strategies for extracting link predictions from LLMs, providing insights into constructing effective prompts for traceability tasks.

This promise warrants a closer look at the traceability challenges that LLM properties may address. Traceability faces three persistent hurdles: the semantic gap between natural-language artifacts and source code, the scale of modern projects [79], and the need for links that auditors can justify. First, state-of-the-art LLMs can accept context windows ranging from roughly 128 000 to two million tokens, so a full requirements specification and a sizable code context can fit in a single prompt, easing the scale barrier [8, 25, 66]. Second, unified embeddings for text and code reduce the semantic gap and yield more precise candidate links than term-matching approaches [5, 35]. Recent work has also explored retrieval-augmented generation (RAG) approaches for traceability: LiSSA [34] demonstrates how combining retrieval mechanisms with generation can improve link recovery while reducing search space, offering a promising direction for scalable traceability solutions. Finally, chain-of-thought prompts and gradient-based token-highlight techniques let a model explain why two artifacts match, providing justifications stakeholders can inspect [57, 64].

Despite these advances, LLM-based *documentation-to-code* traceability remains largely unexplored. Developer documentation is markedly heterogeneous, ranging from README files and usage

tutorials to API docs, which widens the semantic gap and introduces vocabulary drift [75, 76, 95]. Existing LLM traceability studies concentrate primarily on requirements-level links (requirements-to-requirements, requirements-to-code, or requirements-to-models) and rarely assess explanation fidelity or multi-hop chain recovery [5, 36, 39]. By contrast, our work targets developer-facing documentation (API references, tutorials, function-level docstrings, and TransArc’s sentence-level architectural descriptions) to assess how well LLMs support the documentation-to-code traceability tasks developers encounter in day-to-day maintenance. Practitioners also caution that the opaque reasoning of current models complicates certification in audit-heavy domains [79]. We therefore perform a systematic investigation to clarify the strengths and limitations of LLMs before they can be trusted for documentation-centric traceability.

3 METHODOLOGY

In this section, we describe the methodology employed to address our research questions. We discuss the criteria and process for selecting suitable LLMs for evaluation. We then explain how we create and structure our evaluation datasets, and conclude by describing the experimental design used to assess each LLM’s performance.

3.1 LLM Selection

Evaluating the software traceability capabilities of LLMs requires selecting models that represent the current state-of-the-art while allowing for fair assessment on data they have not been trained on. Our selection process focused on identifying LLMs meeting several key criteria:

- *Strong Performance*: Demonstrates capabilities in complex reasoning and code-related tasks, essential for understanding documentation and source code relationships.
- *Large Context Window*: Ability to process substantial amounts of text (documentation and code snippets) simultaneously, crucial for traceability, which often requires broad context.
- *Cost-Effectiveness Features*: Mechanisms like API caching support to mitigate the costs associated with potentially large inputs and iterative analysis common in software engineering tasks.
- *Recent and Verifiable Knowledge Cutoff*: A well-defined training data cutoff date. This ensures we can select evaluation projects created after the model’s training window, preventing data contamination.

Based on these criteria, we selected the following three LLMs:

- **Claude 3.5 Sonnet (claude-3-5-sonnet-20240620)**: Chosen for its state-of-the-art performance at the time of evaluation, particularly its reported strengths in reasoning and coding tasks [8], combined with a large 200k token context window. It has an April 2024 knowledge cutoff.
- **OpenAI GPT-4o (gpt-4o-2024-08-06)**: Selected as OpenAI’s flagship model at the time, known for its strong general performance across various tasks, including code. It offers a 128k token context window and specific optimizations for efficiency [67]. It has an October 2023 knowledge cutoff.
- **OpenAI o3-mini (o3-mini-2025-01-31)**: We also later included OpenAI’s o3-mini to evaluate an LLM specifically highlighted for advanced reasoning capabilities, a trait potentially beneficial for software engineering analysis, while also being designed for cost-efficiency [68, 69]. It features a large 200k token context window. It also has an October 2023 knowledge cutoff like GPT-4o.

Model availability during revision. During the revision window, Claude 3.5 was retired by the provider; therefore, for the added Established Projects experiment reported in Table 5, we used

Claude 4 Sonnet (claude-sonnet-4-20250514). All other experiments (RQ1 on unseen datasets, RQ2, and RQ3) use Claude 3.5 Sonnet.

3.2 Project Selection

To ensure a robust evaluation, we constructed datasets comprising two distinct categories of software projects: (1) a set of *Unseen Projects* to control for data leakage, and (2) a set of *Established Projects* to evaluate generalizability at scale.

3.2.1 *Unseen Projects (Validity Focus)*. To evaluate the performance of LLMs in documentation-to-code traceability tasks, we required datasets containing software projects with well-documented codebases that were unlikely to be part of the selected LLMs' training data. Given that the latest of our selected LLMs has a knowledge cutoff date of April 2024, we focused on projects created after May 1, 2024, to minimize the chance of inclusion in the LLMs' training data. We also prioritized projects based on their popularity (as measured by GitHub star count) to ensure they were more recognized, while placing no constraints on programming language or application domain to allow diversity in the evaluation.

To do this, we used the SEART GitHub search tool [80, 81], which offers advanced filtering options not readily available in GitHub's native search. We filtered projects created after May 1, 2024, and sorted them by descending star count, reflecting their popularity and likelihood of active maintenance. Starting with the most popular projects, we manually reviewed their documentation for clarity, comprehensiveness, and strong links to code elements, examining a total of 128 repositories. From this process, we selected the following two projects because they best satisfied our selection criteria: recent creation date, high popularity, diversity of domain/language, and clear, comprehensive documentation with strong links to code elements:

- **Unity Catalog**¹: An open-source, multimodal catalog that unifies governance across varied data and AI assets. Its repository features structured documentation on supported data formats, API usage, and security configurations—ideal for testing detailed, code-linked documentation.
- **Crawl4AI**²: A web crawler and scraper optimized for AI-centric tasks, offering features like concurrent crawling and metadata extraction. The project's documentation includes extensive usage examples and architecture notes, providing rich opportunities to evaluate how LLMs handle diverse documentation styles when tracing code elements.

3.2.2 *Established Projects (Generalizability Focus)*. To evaluate our methods on a wider range of domains and coding styles, we also considered a set of Established Projects. At the project level, we draw from two existing benchmarks. First, we use CoDocBench [71] as a pool of candidate Python libraries. We selected CoDocBench because it is a multi-project benchmark that directly pairs developer-written natural-language documentation, specifically function docstrings, with the corresponding code. This makes it well suited for evaluating one important class of developer documentation-to-code traceability at scale. We do not treat CoDocBench's function-level docstrings as representative of the full spectrum of documentation artifacts discussed in the introduction. Rather, within our broader evaluation design, CoDocBench serves as a scalable benchmark for function-level developer documentation, while the unseen projects and TransArc cover richer API/tutorial-style and architecture-facing documentation. CoDocBench also provides substantial project diversity at scale (200 open-source Python projects), enabling a stronger generalizability assessment within this function-level developer-documentation setting.

¹<https://github.com/unitycatalog/unitycatalog>

²<https://github.com/unclecode/crawl4ai>

CoDocBench aggregates function-level docstrings and code from 200 open-source projects. We start from its public test split because it already provides a diverse sample of mature libraries while keeping the corpus size manageable for our one-to-many traceability strategy, in which each documentation segment is compared against all candidate artifacts within its project (see Section 3.6). Using the entire CoDocBench corpus would either exceed LLM context windows or require aggressive batching of artifacts, which would substantially change the evaluation setting. Second, we include two Java microservice applications, MediaStore and TeaStore, from the TransArc benchmark [33]. These systems provide sentence-level architectural documentation and curated file-level links.

TransArc includes additional systems beyond MediaStore and TeaStore. We restrict our study to these two because our evaluation setting is *within-project one-to-many*: each documentation segment is scored against *all* candidate artifacts in the project (Section 3.6). For the remaining TransArc systems, the candidate pool is substantially larger (on the order of millions of tokens). Running the same “compare against all artifacts” protocol would exceed current LLM context limits and would require batching or otherwise restricting candidates, which would change the evaluation setting and reduce comparability with our other datasets. We therefore leave these larger systems to future work designed for repositories that exceed LLM context windows.

From the CoDocBench test split, we then select only those projects with at least 20 docstrings, which in our preprocessed dataset also implies at least 20 function-level code artifacts per project. This ensures that each repository offers a non-trivial one-to-many search space. This results in 25 widely used Python libraries, including Pandas, Matplotlib, and PyTorch. Together with MediaStore and TeaStore from TransArc, these choices define our set of 27 Established Projects. Section 3.5 explains how we derive traceability datasets from these projects and reports the resulting numbers of documentation segments and artifacts.

3.3 Traceability Task Definitions

Before detailing our dataset construction methodology, we formally define the traceability tasks evaluated in this study and clarify key terminology. In the broader software engineering domain, *trace link recovery* refers to the goal of recovering missing or broken links between software artifacts [9, 37, 56]. In this work, we focus on *trace link identification*, the specific task of predicting whether a link exists between a documentation segment and a code artifact. Within this task, we distinguish between two fundamentally different problem formulations:

Independent Trace Links. An *independent trace link* is a direct semantic relationship between a documentation segment and a code artifact. Given a documentation segment d and a set of candidate artifacts $\{c_1, c_2, \dots, c_n\}$, the task is to identify all artifacts that d references or describes. Crucially, these links are treated as independent: correctly identifying one link does not depend on identifying another. For instance, documentation describing a crawler’s concurrency setting may independently link to both the `Crawler.__init__` method and the `Crawler` class—each link is evaluated separately. This task is the focus of **RQ1**.

Multi-Step Trace Chains. A *trace chain* is an ordered sequence of dependencies connecting a documentation segment to a target artifact through intermediate elements: $d \rightarrow c_1 \rightarrow c_2 \rightarrow \dots \rightarrow c_n$. Using the same example, the documentation traces through a chain: `docs/config.md` $\rightarrow$ `Crawler.__init__` $\rightarrow$ `Crawler.DEFAULT_CONCURRENCY`, where the constructor serves as an intermediate step to the default value constant. Unlike independent links, chain correctness requires identifying all elements *and* their sequential order, a property that introduces unique challenges we analyze in Section 7.3. This task is the focus of **RQ3**.

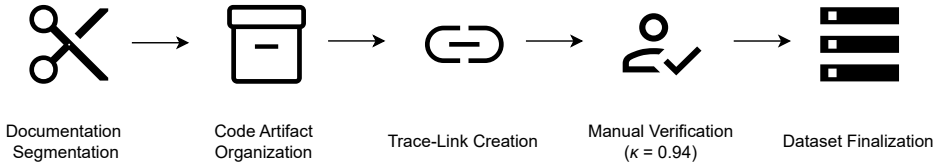


Fig. 1. Five-step dataset-creation pipeline for unseen projects (Unity Catalog and Crawl4AI).

3.4 Manual Dataset Construction (Unity Catalog & Crawl4AI)

To create our ground truth dataset, establishing verified links between documentation and code, we followed a five-step process (shown in Figure 1). This process, detailed below, was designed to ensure the resulting dataset accurately reflects the relationships within the selected software projects and provides a reliable basis for evaluating LLM traceability performance. We illustrate these steps using a simplified, running example based on configuring a web crawler component, similar in nature to elements found in our selected project Crawl4AI.

Running Example Scenario:

```

### Setting Concurrency Level

To control the number of simultaneous requests the crawler
makes, use the `max_concurrent_requests` setting in your
Crawler configuration dictionary. A higher value increases
speed but also resource usage. The default is 5.

Example Usage:
```python
crawler_settings = {
 "start_url": "https://example.com",
 "max_concurrent_requests": 10
}
my_crawler = Crawler(config=crawler_settings)

Newsletter Signup

Stay informed about new features by signing up on our website.
This step is optional and does not affect crawler settings.

```

Fig. 2. Illustrative documentation with two segments. The first segment (above the line) discusses concurrency settings; the second (below the line) briefly mentions a sign-up flow that does not map to any code artifact.

Figure 2 shows two consecutive segments from a hypothetical docs/config.md file. Segment A (“Setting Concurrency Level”) explicitly references how to configure the crawler, while Segment B (“Newsletter Signup”) is unrelated to the crawler’s code and thus has no trace links. Figure 3 presents the corresponding Python code snippet.

```
class Crawler:
 DEFAULT_CONCURRENCY = 5 # Default worker count

 def __init__(self, config):
 self.settings = config
 self.concurrency = self.settings.get(
 "max_concurrent_requests",
 Crawler.DEFAULT_CONCURRENCY
)
 # ... other initialization ...

 def run(self):
 # uses self.concurrency to manage workers...
 pass
```

Fig. 3. Illustrative Python code for the Crawler class, which is referenced by Segment A.

**Step 1: Documentation Segmentation.** The first step involved partitioning the project’s documentation into manageable units, which we term **segments**. A segment is a semantically coherent portion of text, typically discussing a specific feature, API element, configuration option, or a distinct usage scenario. We identified segment boundaries often corresponding to section headings, logical breaks between topics, or distinct code examples. The goal was to create segments focused enough to allow for precise traceability to code, while retaining sufficient surrounding text to provide immediate context [92].

To achieve this, we first performed an automated segmentation by splitting each Markdown file based on its top-level and subsection headings (e.g., #, ##, ###) [56]. However, this automated approach alone was insufficient, as it could separate semantically dependent content or include irrelevant sections. Therefore, we conducted a manual refinement pass guided by two primary rules: (1) *Merging for Context*: Segments that were not self-contained (e.g., a code snippet separated from its explanatory text) were merged with their surrounding context to form a single cohesive unit. (2) *Filtering for Relevance*: Segments lacking any plausible link to the codebase, such as promotional text, general contact information, (e.g., the “Newsletter Signup” in the running example), were discarded. This process ensured that every segment in the final dataset represented a coherent unit of documentation with the potential for traceability.

*Running Example:* In Figure 2, the content labeled “Setting Concurrency Level” was extracted as Segment A. Because it explicitly mentions max\_concurrent\_requests and a crawler usage example, we preserved it for further analysis. Meanwhile, the “Newsletter Signup” text (Segment B) was discarded as it did not discuss any relevant information.

**Step 2: Code Artifact Organization.** Concurrently, we identified and cataloged relevant code elements, termed **code artifacts**. We established the **granularity** (level of detail) for our analysis by focusing on code artifacts typically involved in the software’s architecture, feature-set, and public interface, as these are most likely to be referenced or described in documentation [56]. To ensure a representative yet focused candidate set, we applied a filtering heuristic. We included all primary classes, methods, and attributes while excluding noise such as (1) internal helper functions (e.g., private methods not intended for external use), (2) non-executable configuration files, and (3) generic boilerplate or scaffolding code. This filtering ensures the LLM is evaluated on its ability to map documentation to the core application logic rather than searching through irrelevant utility scripts. Furthermore, to ensure validity, we verified that every artifact explicitly mentioned in the documentation was present in this final candidate set.

Regarding granularity, it is important to distinguish between public interface elements and internal logic. While our dataset includes “Statement-level” artifacts in the form of class attributes and constants (which are frequently referenced in documentation), we deliberately excluded finer-grained implementation details like individual statements buried within methods or local variables. This decision is based on the observation that documentation usually describes what components do or how to use them at an interface level, rather than detailing internal statement-by-statement logic; including such low-level details would likely add significant noise with few corresponding documentation links, a common consideration in traceability link recovery [56].

It is important to emphasize that defining these source units (documentation segments) and target units (code artifacts) upfront is not a simplification designed to make the task easier for LLMs, but rather a *methodological necessity* for conducting valid traceability evaluation. Without clearly defined boundaries on both ends of potential trace links, there exists no objective ground truth against which to evaluate LLMs’ and baselines’ performance. Different granularity choices would produce fundamentally incomparable results: an LLM mapping entire files to packages cannot be meaningfully compared against one mapping paragraphs to methods. This practice of establishing the artifact space prior to evaluation is well-established in traceability research [9, 52, 56], where researchers consistently define document units and code units before conducting experiments.

Each identified artifact was stored with its type (Class, Method, Function, Attribute, etc.), full path location, granularity levels (Class-level, Method-level, and Statement-level), and the source code content. Note that there is a subtle difference between artifact types and their granularities. For instance, a Python standalone function and a class method, are both considered to be Method-level. The same applies to a class attribute and a code statement, which we both consider to be Statement-level.

*Running Example:* From the example code, we identify the following artifacts matching our granularity:

- Crawler (Type: Class, Location: `src/crawler.py`, Granularity: Class-level)
- `Crawler.__init__` (Type: Method, Location: `src/crawler.py`, Granularity: Method-level)
- `Crawler.DEFAULT_CONCURRENCY` (Type: Attribute, Location: `src/crawler.py`, Granularity: Statement-level)

**Step 3: Trace Link Creation.** Next, we manually examined each documentation segment against the extracted code artifacts to establish **trace links**. For every segment, we performed a multi-faceted analysis by:

- **Identifying Explicit Mentions:** Searching for direct references by name, such as “Crawler constructor”, “`max_concurrent_requests` setting”, or “`__init__` method.”

- **Analyzing Implicit Relationships:** Examining textual descriptions and usage examples to identify code artifacts that are used, configured, or demonstrated even if not explicitly named in the narrative.
- **Characterizing and Justifying:** For each potential link, documenting the nature of the relationship (e.g., “describes configuration of”, “provides default value for”, “example instantiates”) and providing a concise justification based on evidence from the documentation and corresponding code.
- **Identifying Trace Chains:** Recognizing sequences where a documentation segment links to one artifact which, in turn, relates to another (for example, a segment might reference a configuration parameter in a method that falls back to a class constant). These chains represent a pathway of understanding from the documentation to deeper implementation details.

*Running Example:* For our example segment, trace links would be created to:

- `Crawler.__init__`: Relationship: “Describes configuration parameter (`max_concurrent_requests`) used by this constructor”. Justification: The documentation explains the parameter, and the example shows it passed via `crawler_settings` to the `Crawler` constructor, which then assigns it internally.
- `Crawler.DEFAULT_CONCURRENCY`: Relationship: “Specifies the default value for the concept implemented by this constant”. Justification: The text states “The default is 5”, directly corresponding to the constant used as a fallback in `__init__`.
- `Crawler` (Class): Relationship: “Provides a usage example instantiating this class”. Justification: The code example explicitly creates an instance with `my_crawler = Crawler(...)`.

A trace chain here is: Segment A  $\rightarrow$  `Crawler.__init__`  $\rightarrow$  `Crawler.DEFAULT_CONCURRENCY`.

**Step 4: Quality Control & Verification.** All proposed links underwent a manual verification pass to confirm they were valid and accurately reflected the relationships described in the text. Inter-rater agreement between the first and second authors was Cohen’s  $\kappa = 0.94$ , indicating almost-perfect agreement [50]. Disagreements were then discussed and resolved to reach a consensus. Each proposed trace link was reviewed against several criteria:

- **Appropriate Granularity:** We verified that each link was established at the most appropriate level of abstraction. While each artifact has a pre-defined type, annotators must still choose *which* artifact to link to, and this choice can be made at an inappropriate level. This step caught two types of human annotation errors: (1) *Over-generalization*, where documentation about a specific method was incorrectly linked to the entire parent class; and (2) *Over-specification*, where documentation describing a class’s overall purpose was linked only to one of its minor attributes. This verification ensured the precision of our ground truth by matching links to the granularity implied by the documentation content.
- **Justification Clarity & Accuracy:** Is the written explanation for the link clear, technically correct, and well-supported by the content of the segment and the artifact?
- **Trace Validity:** Does the link represent a meaningful semantic connection relevant for software understanding or maintenance (e.g., configuration, usage example, implementation detail description)? Are identified trace chains logical pathways?
- **Completeness Check:** Considering the segment, are there any obvious, significant links (especially implicit ones revealed by examples) that were missed?

Links deemed invalid were either corrected or removed. This review also ensured that only the publicly relevant portions of code (e.g., classes, methods, class-level attributes, etc.) were linked, thus avoiding noise from private implementation details or purely internal variables.

*Running Example:* In the verification process, we would check if the links created in Step 3 for the concurrency segment are accurate (e.g., does `__init__` actually use the parameter? Is 5 truly the default value constant?). We would assess if the justifications are understandable and confirm that the granularity is correct (linking to the method and attribute, not just arbitrary lines).

Finally, based on these verified links, the first author manually authored reference explanations describing the specific nature of each relationship (e.g., “implements interface,” “configures parameter”) by analyzing the code logic. These explanations serve as the ground truth for evaluating the semantic quality of LLM-generated insights.

**Step 5: Dataset Structure Finalization.** Finally, the verified data was organized into a structured format suitable for experimental use. Each record in the dataset corresponds to one documentation segment. It contains the segment’s text, its metadata (source file, location), and a list of all verified code artifacts traced to it. For each linked artifact, the record stores detailed information derived from the previous steps: the artifact’s identifier (e.g., `src/crawler.py::Crawler.__init__`), its type, location, source code, the relationship description, the granularity level of the trace, and any relevant trace chain information. This structured format facilitates parsing the data for experiments and enables various types of analysis on the nature and distribution of trace links.

*Running Example:* The final record for our example segment would bundle the Markdown text, its metadata, and an array containing entries for the links to `Crawler.__init__`, `Crawler.DEFAULT_CONCURRENCY`, and `Crawler`, each with their respective justifications and other details gathered in Steps 2-4. Figure 4 illustrates the structure of a dataset entry, using our running example.

We repeated this workflow for all documentation segments across both repositories, yielding two distinct datasets. Table 1 summarizes the key characteristics of the two unseen datasets. The per-project characteristics of the 27 Established Projects are reported in Appendix A.1. In *Crawl4AI*, we obtained 112 documentation segments and 29 code artifacts, resulting in a total of 645 trace links. In contrast, *Unity Catalog* provided a more focused dataset with 32 segments and 76 code artifacts, yielding 155 trace links. On average, the *Crawl4AI* dataset produced about 5.8 traces per segment compared to 4.8 in *Unity Catalog*. In terms of granularity, *Crawl4AI* exhibits a diverse distribution with 464 class-level, 98 method-level, and 83 statement-level links. For *Unity Catalog*, however, only 13 class-level and 142 method-level trace links were identified; no statement-level links were found, which is attributable to the nature of its API documentation that predominantly references higher-level code constructs. This variation in scale and density across the two datasets provides a rich basis for evaluating LLM traceability capabilities.

Intuitively, how hard this task is for an LLM depends on the properties of the datasets and how we frame traceability. For each documentation segment, the model sees the full set of candidate code artifacts for the project and must decide which ones are linked. The difficulty therefore depends on both how many artifacts are available as candidates and how many of them are actually linked to each segment.

To make this concrete, we characterize task complexity along two complementary axes, following prior work on traceability dataset characterization [13, 40]. The first axis captures *search sparsity*: the *link density* measures the proportion of all possible document–code pairs that constitute true links ( $L/(N \times M)$ ). Lower density indicates a sparser search problem where correct links are rarer. The second axis captures *subset selection difficulty*: we define a combinatorial complexity index  $C_{avg} = \log_2 \binom{M}{\lceil \bar{k} \rceil}$ , which approximates the number of bits required to identify the correct subset of linked artifacts for a typical segment. This follows from a simple counting argument: if a segment has  $k$  true links among  $M$  candidate artifacts, then there are  $\binom{M}{k}$  distinct  $k$ -sized subsets that could be predicted. Using  $k \approx \lceil \bar{k} \rceil$ ,  $\log_2 \binom{M}{\lceil \bar{k} \rceil}$  captures the information needed to uniquely specify the

```

{
 "document": {
 "text": "### Setting Concurrency Level\n\nTo control the number of simultaneous
 ...\nExample Usage:\n```python\ncrawler_settings = {\n \"start_url\": \"
 https://example.com\", \n \"max_concurrent_requests\": 10\n}\nmy_crawler =
 Crawler(config=crawler_settings)\n```",
 "location": "docs/config.md",
 "type": "Configuration Section"
 },
 "artifacts": [
 {
 "title": "Crawler.__init__",
 "location": "src/crawler.py",
 "content": "def __init__(self, config):\n self.settings = config\n self.
 concurrency = self.settings.get(...)",
 "type": "Method",
 "relationship": "Describes configuration parameter (max_concurrent_requests)
 used by this constructor.",
 "traceability_granularity": "Method-level",
 "trace_chain": "docs/config.md -> Crawler"
 },
 {
 "title": "Crawler.DEFAULT_CONCURRENCY",
 "location": "src/crawler.py",
 "content": "DEFAULT_CONCURRENCY = 5",
 "type": "Attribute",
 "relationship": "Specifies the default value for the concept implemented by
 this constant.",
 "traceability_granularity": "Statement-level",
 "trace_chain": "docs/config.md -> Crawler.__init__ -> Crawler.
 DEFAULT_CONCURRENCY"
 },
 {
 "title": "Crawler",
 "location": "src/crawler.py",
 "content": "class Crawler:\n DEFAULT_CONCURRENCY = 5 ...",
 "type": "Class",
 "relationship": "Provides usage example instantiating this class.",
 "traceability_granularity": "Class-level",
 "trace_chain": "docs/config.md -> Crawler"
 }
]
}

```

Fig. 4. Illustrative structure of a dataset entry using the running example.

Table 1. Dataset characteristics for unseen projects (Unity Catalog and Crawl4AI)

Characteristic	Crawl4AI	Unity Catalog
Document Segments (N)	112	32
Code Artifacts (M)	29	76
Total Traces (L)	645	155
Avg. Traces/Segment ( $\bar{k}$ )	5.76	4.84
Link Density	19.9%	6.4%
Combinatorial Complexity (bits)	18.9	24.0
<b>Granularity Distribution</b>		
Class-level	464	13
Method-level	98	142
Statement-level	83	0

correct subset among these alternatives. Higher values indicate more ways for a predicted subset to be partially incorrect. These two axes are complementary: density reflects class imbalance (affecting precision), while combinatorial complexity reflects the challenge of identifying multiple correct targets from a large candidate set (affecting recall). For instance, Unity Catalog presents a sparser search problem (6.4% density) than Crawl4AI (19.9%), but Crawl4AI requires identifying more links per segment, yielding similar combinatorial complexity ( $\sim 19\text{--}24$  bits).

### 3.5 Automated Dataset Derivation (CoDocBench)

This subsection focuses on the CoDocBench-derived portion of the Established Projects dataset; the two TransArc systems reuse the existing TransArc traceability ground truth and are only converted into our unified JSON format.

The original CoDocBench dataset is structured for alignment tasks, providing pairs of code and documentation changes across version history (e.g., `diff_code`, `diff_docstring`) [71]. To adapt this for our documentation-to-code traceability evaluation, we processed the raw data to create a static retrieval benchmark. The derivation involved four key steps:

- **Snapshot Extraction:** We transitioned from evolution-based data to a static traceability view by extracting the latest (`docstring`, `code`) pair for each function, effectively flattening the version history into a current-state snapshot of the codebase.
- **Project Filtering:** We excluded projects with fewer than 20 document segments. This threshold ensures that the repository possesses sufficient complexity for the one-to-many retrieval task; projects with fewer artifacts often result in trivial retrieval scenarios where the search space is negligibly small.
- **Context Reconstruction:** While the raw dataset provides file paths (e.g., `file_path`), it lacks the explicit directory hierarchy required for the LLM’s structural understanding. We reconstructed the full directory trees for each selected project to provide the necessary context, matching the format used in our manually constructed datasets.
- **Deduplication and Metadata:** We collapsed multiple entries for the same function to the latest pair. To handle duplicate docstrings (e.g., identical documentation for overloaded functions), we appended specific project and function metadata (Project, Function, File) to ensure every documentation segment in our evaluation was uniquely identifiable.

Applying these steps to the CoDocBench test split yields a CoDocBench-derived traceability dataset with **1,820 documentation segments** and **1,820 function-level code artifacts** across 25 projects. For the two TransArc projects (MediaStore and TeaStore), we reuse the sentence-level



```

cached_content = {
 "available_artifacts": [
 {
 "artifact_id": artifact["artifact_id"],
 "title": artifact["title"],
 "location": artifact["location"],
 "content": artifact["content"]
 }
 for artifact in request["artifact_list"]
],
 "directory_tree": request["directory_tree"]
}

prompt_dictionary = {
 "task": "Documentation to Code Traceability Analysis",
 "description": "Analyze documentation to identify relevant
code artifacts and relationships",
 "document": {
 "text": "<documentation_text>",
 "location": "<file_location>"
 },
 "instructions": {
 "steps": [
 "1. Analyze specific text for traces",
 "2. Identify explicit/implicit mentions",
 "3. Explain relationships",
 "4. Construct trace chains"
]
 }
}

```

Fig. 5. Simplified prompt structure for documentation-to-code traceability.

documentation and curated file-level links provided by the benchmark [33] and convert them into the same JSON schema as our CoDocBench-derived projects. These two systems contribute an additional **48 documentation segments** and **160 code artifacts**. Together, the CoDocBench and TransArc portions form the Established Projects dataset used in our experiments, comprising **1,868 documentation segments** and **1,980 code artifacts** drawn from **25 CoDocBench projects** and **2 TransArc projects** (27 repositories in total). Per-project statistics are reported in Appendix A.1.

Across these 27 projects, *link density* ranges from 0.33% to 21.2%, while our combinatorial complexity index  $C_{avg}$  ranges from 4.5–8.2 bits for most CoDocBench libraries, 6.7 bits for MediaStore, and 93.2 bits for TeaStore (Table 16). This highlights that the Established Projects dataset spans both sparse single-link settings and highly multi-link configurations such as TeaStore.

### 3.6 Experimental Setup

**Overview.** In this section, we describe the experimental methodology used to evaluate the traceability capabilities of the selected LLMs. We outline the dataset preparation, explain the evaluation strategy (including data ordering and prompting approach), define the baselines chosen for comparison, and clarify the metrics and additional analyses performed.

**Datasets.** We conducted our evaluation across the two categories of datasets described in Section 3.2. This includes the Unseen Projects (Crawl4AI and Unity Catalog), which together provide 144 documentation segments, and the Established Projects, which combine 25 CoDocBench libraries with 2 TransArc systems and contribute an additional 1,868 documentation segments across 27 repositories. Each documentation segment is assessed individually against all available code artifacts within its own project (one-to-many strategy; other strategies are discussed in Section 7.1).

**Evaluation Procedure.** To provide more robust performance estimates, we repeat each experiment five times. Each experiment involves:

- Randomizing the order in which documentation segments are processed.
- Providing each segment with the same project-level candidate artifact set under the one-to-many strategy.
- Feeding each segment (with its list of candidate artifacts) to the LLM and collecting its predicted links.
- Aggregating all predictions across segments to compute precision, recall, and F1 for that run.

These steps help account for variation across repeated runs while preserving a comparable candidate search space across models. We separately analyze candidate-list position in Section 4.3.

**Prompt Structure.** We created a structured prompt for the documentation-to-code traceability task, as depicted in Figure 5. Unlike traditional single-text prompts, our format uses a structured input schema to provide the LLM with separate, well-organized fields for instructions, context (available code artifacts), and the documentation segment under analysis [7, 65]. This design ensures clarity, repeatability, and control over the LLM’s attention. The prompts explicitly instruct the LLMs to identify both explicit and implicit references within the given documentation segment, restricting trace analysis solely to the provided artifacts. Each prompt contains two main components

- `cached_content`: A JSON object listing available artifacts (their titles, content, and locations) and the directory tree structure of the repository, providing relevant context.
- `prompt_dictionary`: Detailed instructions directing the LLMs’ analysis, including how to identify relationships, differentiate between explicit and implicit references, and correctly format trace chains (e.g., `docs.md` → `ClassA` → `MethodB`).

It is important to note that Figure 5 shows a simplified view for clarity and brevity. The actual prompt employed is considerably more elaborate, featuring numerous detailed steps (e.g., specifying granularity rules, defining relationship types, formatting trace chains precisely) and strict output schema enforcement, as necessitated by the complexity of the traceability task. Complete versions of the prompts are provided in the Appendix.

**LLM Settings and API Details.** We evaluated three LLMs: *Claude 3.5 Sonnet*, *GPT-4o*, and *o3-mini*, as described in detail in Section 3.1. Default parameters were employed in each case, except that the parameter `reasoning_effort` was explicitly set to `high` for *o3-mini* to maximize its reasoning performance. For the Established Projects experiment (Table 5), we utilized the newer Claude 4 Sonnet, as the 3.5 version was deprecated by the provider during the study. API requests were made directly through the official providers (Anthropic and OpenAI). For efficiency, we leveraged

built-in request caching: OpenAI enables caching by default for all API calls, while for Anthropic, it was explicitly enabled for our experiments.

**Baseline Methods.** We compared the LLM results against three widely used baseline methods in software traceability:

- **TF-IDF:** computes cosine similarity between TF-IDF vectorized documentation segment and code artifact [9].
- **BM25:** employs a probabilistic token-matching approach (Okapi BM25) to rank code artifacts given a documentation segment [77].
- **CodeBERT:** utilizes pretrained CodeBERT embeddings trained specifically on software texts and code. Due to its 512-token limit, CodeBERT applies a sliding-window approach for longer inputs, averaging embeddings across windows before computing cosine similarities [31].

Each baseline method outputs a similarity score between a documentation segment and a code artifact—typically in the range  $[0,1]$  for TF-IDF and CodeBERT (cosine similarity), and a non-normalized score for BM25. To convert these continuous scores into binary trace links, we perform threshold tuning for each method individually. Specifically, we conduct a grid search in 0.05 increments and select the threshold that maximizes F1-score on the evaluation set. This represents an optimistic estimate of baseline performance, since the thresholds are selected post hoc based on the same data being evaluated.

**Evaluation Metrics for One-to-Many Traceability.** Evaluating a one-to-many traceability task requires a precise definition of how predictions are mapped to standard Information Retrieval metrics. We formally define the problem as a set of  $N$  documentation segments and  $M$  candidate code artifacts. We adopted a “flattening” approach, decomposing the evaluation into  $N \times M$  independent binary classification decisions. For every possible (documentation, code) pair in the search space, we determine:

- **True Positive (TP):** A link predicted by the LLM that exists in the ground truth.
- **False Positive (FP):** A link predicted by the LLM that does not exist in the ground truth.
- **False Negative (FN):** A link not predicted by the LLM that exists in the ground truth.

We sum these counts across the entire dataset to calculate the overall Precision, Recall, and F1-score. This methodology ensures the models are robustly penalized for incorrect predictions across the entire search space of candidate artifacts.

**Metrics and Granularity Analysis (RQ1).** We measure standard Information Retrieval metrics: precision, recall, and F1-score [9, 56]. Additionally, we report metrics grouped by artifact granularity (class-level, method-level, and statement-level), providing deeper insight into how performance varies with artifact detail [12, 52]. These granular metrics help clarify the strengths and weaknesses of each approach at different traceability levels.

**Relationship Explanation (RQ2).** For RQ2, we evaluated the quality of relationship explanations provided by the LLMs. Given the substantial volume of generated links, we used an LLM-based evaluation approach (LLM as a judge) [4, 94], classifying explanations into three accuracy categories: *correct*, *partially correct*, or *incorrect*. To validate the reliability of this automated evaluation, a representative subset was manually reviewed and verified. More details on this are found in Section 5.

**Trace Chain Analysis (RQ3).** For RQ3, we analyzed trace chains generated by LLMs, comparing predicted chains against ground truth. We classified predicted chains based on how closely they

matched actual trace pathways, using defined correctness categories: *complete match*, *partial match* (with subcategories), or *incorrect*. Section 6 describes these categories and analysis details further.

### 3.7 Reproducibility and Ethical Considerations

To ensure the reproducibility of our study, all experiments were implemented in Python 3.11. The code, datasets, analysis scripts, and prompt templates used are publicly available in our replication package<sup>3</sup>. This package includes detailed instructions for reproducing our results, and we have documented our experimental procedures following standard research practices.

Throughout the study, we adhered to ethical guidelines. This included respecting the licenses of the software projects used in our datasets, complying with the terms of service and usage policies provided by the LLM vendors, and ensuring that no sensitive or personal data was included or exposed during dataset creation or experimentation.

## 4 RQ1: TRACE LINK ACCURACY

For RQ1, *How accurately can LLMs identify trace links between documentation segments and their corresponding code elements?*, we aim to examine if LLMs can establish traceability effectively between document and code elements. We present our motivation, approach, and results.

### 4.1 Motivation

While LLMs demonstrate strong capabilities in understanding both natural language and code [17, 48], their ability to create accurate trace links between software documentation and source code has not been systematically studied. Establishing these links is essential for core software engineering tasks like change impact analysis, program comprehension, and maintaining documentation consistency with evolving code [23, 89]. As discussed in Section 2.3, traditional methods often struggle with this task. LLMs offer a promising alternative due to their advanced semantic understanding [29]. Therefore, assessing their accuracy is essential to determine if LLMs are useful for improving documentation to code traceability.

### 4.2 Approach

To answer this RQ, we evaluate the traceability capabilities of the selected LLMs. For our primary evaluation on unseen datasets, we use Claude 3.5 Sonnet, GPT-4o, and o3-mini. For the generalizability experiments on the Established Projects dataset (25 CoDocBench libraries plus 2 TransArc systems), we substitute Claude 3.5 with the newer Claude 4 Sonnet, since the 3.5 model was retired by the provider during our revision window. Each LLM is asked to analyze documentation segments to identify relevant code artifacts. This analysis considered both explicitly named artifacts mentioned in the text and implicitly related artifacts inferred from contextual clues, such as code usage examples, presented within the documentation.

For each documentation segment, we provided the LLMs with relevant contextual information: the document text itself, its location within the file. For the code artifacts, we provided the LLMs with their location within the directory, the code itself, and the project's directory structure. This rich context enabled the LLMs to understand both local and project-wide relationships.

Our evaluation process incorporated multiple measures to ensure reliable results. We repeated each experiment five times and randomized the order in which documentation segments were processed across runs. The candidate artifact set for each project was kept consistent across models and runs, and we separately analyze whether candidate-list position affected trace recovery in Section 4.3. To compare fairly, we gave both the LLMs and the baseline methods the same inputs:

<sup>3</sup><https://github.com/Alor-e/evaluating-llm-doc-code-traceability>

the document segments and the list of code artifacts to check against using a one-to-many matching strategy. We assessed the overall accuracy of trace link identification using standard Information Retrieval metrics: Precision, Recall, and F1-score.

To further understand the LLMs' performance, we employed a mixed-methods approach on the manually constructed datasets.

For all incorrect trace links (False Positives), we performed a manual qualitative analysis. The first author conducted the initial review of the false positives to identify the specific reason for each failure (e.g., assuming a link exists based only on a similar name), and the resulting recurring patterns were subsequently reviewed by the second author and grouped into the error categories presented in Table 3.

For missed trace links (False Negatives), we performed a quantitative analysis comparing their characteristics against those of correctly identified trace links to identify factors associated with successful or failed trace identification, examining attributes including document length, presence of code examples, and trace link granularity.

### 4.3 Results

Table 2 presents performance metrics across the selected LLMs, baseline models, and datasets, demonstrating both the strengths and limitations of different approaches. Within each dataset, the best-performing model (confirmed by the *Scott-Knott ESD* test [86, 87]) for each metric (Precision, Recall, F1) is highlighted in bold. For brevity, we omit the baselines' granularity breakdown here, but those detailed results can be found in the Appendix.

**Traceability Performance.** The evaluated LLMs demonstrate overall F1 scores (across all granularities) ranging from 64.2% to 80.4% across both datasets, indicating a moderate to strong performance in identifying traceability links. A key strength of the LLMs lies in their overall precision, which is consistently high (above 87%) and substantially outperforms the baseline methods. However, while overall precision is uniformly high, their overall recall varies significantly (ranging from 47% to 75% across the models and datasets). This divergence in recall is the primary factor driving the differences in their overall F1 scores.

We observe consistent performance patterns among the LLMs across the datasets. Claude Sonnet 3.5 achieves the best F1 score, demonstrating the most effective balance between high overall precision and the highest overall recall among the evaluated LLMs. Conversely, o3-mini achieves the highest overall precision, which could be attributed to its reasoning capabilities. The baseline CodeBERT achieves the highest overall recall, but this is accompanied by very low precision, indicating that it identifies almost all possible links, including a large number of incorrect ones. It is also noteworthy that on the Unity Catalog dataset, the TF-IDF baseline achieves an F1 score competitive with GPT-4o and superior to o3-mini on this specific dataset.

Furthermore, performance varies across the granularity of the code artifacts being traced, especially regarding recall compared to precision. Method-level tracing generally achieves higher F1 scores across the models and datasets compared to class-level tracing (e.g., 97.5%–98.9% vs. 58.2%–71.1% in Crawl4AI), with statement-level performance typically falling in between (e.g., 76.5%–94.0% in Crawl4AI); however, exceptions exist, such as Claude Sonnet 3.5's strong class-level performance in Unity Catalog.

**Analyzing Incorrect Links (False Positives).** Table 3 presents the frequency of error patterns across LLMs and datasets. Note that GPT-4o and o3-mini showed very high precision scores and thus minimal (0 or 1) total false positives. Below, we discuss each error category, arranged from most to least frequent, highlighting specific reasons behind each type of failure.

Table 2. Documentation-to-Code Traceability Performance Across LLMs, Baseline Models and Datasets

Dataset	Model	Precision (%)	Recall (%)	F1 (%)
Crawl4AI	<b>Claude Sonnet 3.5</b>	95.3	68.0	<b>79.4</b>
	Class	93.7	57.3	71.1
	Method	97.8	100.0	98.9
	Statement	98.0	90.5	94.0
	<b>GPT-4o</b>	96.8	53.0	68.5
	Class	96.4	41.7	58.2
	Method	96.4	98.6	97.5
	Statement	99.2	62.2	76.5
	<b>o3-mini</b>	<b>98.5</b>	58.0	73.0
	Class	98.4	45.7	62.4
	Method	97.6	98.4	98.0
	Statement	100.0	79.3	88.4
	<b>TF-IDF</b>	50.3	58.7	54.2
	<b>BM25</b>	33.4	65.0	44.1
	<b>CodeBERT</b>	22.8	<b>87.7</b>	36.2
Unity Catalog	<b>Claude Sonnet 3.5</b>	87.0	74.8	<b>80.4</b>
	Class	97.3	100.0	98.6
	Method	85.8	72.5	78.6
	<b>GPT-4o</b>	99.7	53.2	69.3
	Class	100.0	18.5	30.1
	Method	99.7	56.3	72.0
	<b>o3-mini</b>	<b>100.0</b>	47.3	64.2
	Class	60.0	6.2	11.0
	Method	100.0	51.1	67.6
	<b>TF-IDF</b>	77.6	62.6	69.3
	<b>BM25</b>	12.9	40.0	19.5
	<b>CodeBERT</b>	6.4	<b>100.0</b>	12.0

Note: Bold values indicate best overall performance for each metric within each dataset.

*Implicit Assumption Errors (IAE)* occurred when LLMs inferred relationships without sufficient evidence, often due to reliance on naming conventions or their pretrained knowledge of common software patterns. IAEs were the most frequent errors across all LLMs. For example, Claude Sonnet 3.5 mistakenly assumed a class inheritance between CosineStrategy and ChunkingStrategy classes in the Crawl4AI dataset based purely on naming similarity, even though no explicit inheritance existed in the code.

*Phantom Link (PAL)* errors were generated when LLMs traced documentation to code artifacts referenced implicitly or explicitly in the documentation but not present within the provided code artifact set. Claude Sonnet 3.5 frequently produced PAL errors, such as incorrectly linking documentation to the method NlpSentenceChunking.chunk(), which was mentioned in documentation examples in the Crawl4AI dataset but absent from the provided artifacts. In contrast, GPT-4o and o3-mini rarely encountered this issue.

Table 3. False Positive Error Pattern Analysis Across LLMs and Datasets

Dataset	Model	Error Pattern	Frequency (%)
Crawl4AI	Claude Sonnet 3.5	Implicit Assumption (IAE)	61.3
		Phantom Link (PAL)	30.7
		Architecture Pattern Bias (APB)	0.0
		Implementation Overlink (IOL)	8.0
		<i>Total False Positives</i>	<i>150</i>
	GPT-4o	Implicit Assumption (IAE)	98.2
		Phantom Link (PAL)	1.8
		Architecture Pattern Bias (APB)	0.0
		Implementation Overlink (IOL)	0.0
		<i>Total False Positives</i>	<i>56</i>
	o3-mini	Implicit Assumption (IAE)	52.6
		Phantom Link (PAL)	0.0
		Architecture Pattern Bias (APB)	0.0
		Implementation Overlink (IOL)	47.4
		<i>Total False Positives</i>	<i>38</i>
Unity Catalog	Claude Sonnet 3.5	Implicit Assumption (IAE)	51.9
		Phantom Link (PAL)	15.5
		Architecture Pattern Bias (APB)	32.6
		Implementation Overlink (IOL)	0.0
		<i>Total False Positives</i>	<i>129</i>
	GPT-4o	Implicit Assumption (IAE)	100.0
		Phantom Link (PAL)	0.0
		Architecture Pattern Bias (APB)	0.0
		Implementation Overlink (IOL)	0.0
		<i>Total False Positives</i>	<i>1</i>
	o3-mini	Implicit Assumption (IAE)	0.0
		Phantom Link (PAL)	0.0
		Architecture Pattern Bias (APB)	0.0
		Implementation Overlink (IOL)	0.0
		<i>Total False Positives</i>	<i>0</i>

Note: The frequency shows the proportion of each error type relative to the total false positives for that specific model/dataset combination.

*Architecture Pattern Bias (APB)* errors arose from LLMs' tendency to overgeneralize architectural patterns observed in specific parts of the codebase, applying them incorrectly to unrelated areas. Such errors were only observed in Claude Sonnet 3.5 for the Unity Catalog dataset, where the model incorrectly assumed that all services adhered to a specific architectural pattern (e.g., `ModelService` → `ModelRepository`). Services employing different architectures (e.g., temporary credential services in the dataset) often triggered APB errors.

Table 4. Comparison of Document Characteristics Impact on Trace Identification Across LLMs and Datasets

Dataset	Model	Characteristic	TP	FN
Crawl4AI	Claude Sonnet 3.5	Explicit Mentions (%)	44.6	9.1
		Usage Examples (%)	94.7	97.9
		Avg. Document Length (words)	85.3	79.0
		Class-level (%)	60.9	96.2
		Method-level (%)	22.3	0.0
		Statement-level (%)	16.9	3.8
	GPT-4o	Explicit Mentions (%)	44.8	18.3
		Usage Examples (%)	94.3	97.9
		Avg. Document Length (words)	83.4	82.0
		Class-level (%)	56.7	89.4
		Method-level (%)	28.5	0.5
		Statement-level (%)	14.8	10.1
	o3-mini	Explicit Mentions (%)	49.2	9.3
		Usage Examples (%)	93.8	98.9
		Avg. Document Length (words)	86.1	79.0
		Class-level (%)	56.7	93.1
		Method-level (%)	25.9	0.6
		Statement-level (%)	17.5	6.3
Unity Catalog	Claude Sonnet 3.5	Explicit Mentions (%)	87.9	97.4
		Parameters>Returns (%)	60.4	23.7
		Avg. Document Length (words)	83.2	81.5
		Class-level (%)	11.2	0.0
		Method-level (%)	88.8	100.0
	GPT-4o	Explicit Mentions (%)	96.1	83.7
		Parameters>Returns (%)	68.7	37.7
		Avg. Document Length (words)	89.0	75.7
		Class-level (%)	2.9	14.6
		Method-level (%)	97.1	85.4
	o3-mini	Explicit Mentions (%)	97.5	83.7
		Parameters>Returns (%)	66.4	43.2
		Avg. Document Length (words)	88.8	78.0
		Class-level (%)	1.1	15.1
		Method-level (%)	98.9	84.9

*Implementation Overlink (IOL)* errors resulted when LLMs inappropriately traced documentation to internal implementation details, violating abstraction boundaries. Claude Sonnet 3.5 and o3-mini generated these errors in the Crawl4AI dataset, incorrectly linking documentation segments to private helper methods or internal functions not intended for external documentation reference. GPT-4o notably avoided such errors entirely.

**Analyzing Missed Links (False Negatives).** To understand why LLMs sometimes fail to identify correct trace links (False Negatives), we compared the characteristics of the documentation segments between correctly identified links (True Positives) and missed links (False Negatives). Table 4 presents this comparison across several key characteristics of each dataset. We specifically examined the presence of *explicit mentions* (whether the documentation directly names the code artifact),



*usage examples* (whether there are segments containing code snippets within fenced blocks). We also considered *structured sections* such as Parameters and Returns specific to the Unity Catalog dataset (as commonly found in API-style documentation). Finally, we considered the different levels of granularity of the trace links (i.e., Class-level, Method-level, and Statement-level, where applicable). The results differ notably between the two datasets due to their distinct documentation styles.

*Crawl4AI*. This dataset uses narrative-style documentation, often featuring illustrative usage examples. The results show that explicit mentions of code artifacts greatly influence trace link success: segments with explicit mentions were consistently more successful compared to missed links across the LLMs (45–49% of TPs vs only 9–18% of FNs). Importantly, missed links predominantly occur at the class level (at least 89% of all FNs), highlighting a clear difficulty for the LLMs in identifying class-level information in less structured, narrative-style documentation. Method-level and statement-level links rarely resulted in misses, aligning with the previously observed higher recall at these granularities (as seen in Table 2).

*Unity Catalog*. Unlike Crawl4AI, Unity Catalog features structured API reference documentation without explicit usage examples or statement-level traces. The critical factor influencing traceability is the existence of structured sections (Parameters and Returns) (60–69% of TPs vs 24–43% of FNs). Document segments containing these structured sections are more likely to be linked correctly, while those lacking such structure see considerably higher miss rates.

*Cross-dataset Insights*. The differing patterns in these two datasets suggest that successful traceability heavily relies on the dominant type of contextual cue provided by the documentation. In Crawl4AI (narrative-style documentation), explicit mentions of artifacts enhance recall (due to lower false negatives), especially at higher abstraction levels (i.e., classes). In Unity Catalog (structured API documentation), where explicit mentions are nearly always present, additional structural cues such as parameter and return tables is decisive in correctly establishing trace links.

*Candidate-list Position Analysis*. We further analyzed whether the position of a ground-truth artifact in the candidate list was associated with trace recovery. For each project, we normalized artifact positions and divided the candidate list into early, middle, and late thirds. We then measured traceability performance within each third; Table 17 reports the recall breakdown because recovery varies most clearly along this dimension. Precision remained high across positions for the unseen datasets (91.4%–99.4%), so the main variation was in recovery rather than correctness. The pattern is not uniform across datasets. In the unseen projects, early-position artifacts generally had lower recall than middle or late-position artifacts. In the Established Projects, the CoDocBench subset showed consistently high recall across all positions, while the TransArc systems had much lower recall overall, especially for early and middle positions. This suggests that candidate-list position is associated with trace recovery, but the effect depends strongly on dataset structure and artifact type. In particular, in Crawl4AI, early-position ground-truth links are dominated by class-level artifacts, which Table 4 identifies as difficult to recover.

**Performance on Established Projects (Generalizability).** To assess whether our findings generalize to established codebases, we evaluated performance on the 27 Established Projects dataset (Table 5). The results highlight three key patterns:

1. *Consistency of LLM Dominance*: Consistent with our findings on the unseen datasets, LLMs demonstrate superior performance. o3-mini achieved the highest average F1-score (92.3%), closely followed by GPT-4o (88.1%). Both models significantly outperformed the best baseline, TF-IDF (60.5%), across 26 of the 27 projects. This confirms that the reasoning capabilities observed in our smaller datasets scale effectively to major libraries like Pandas and PyTorch. The contrast with the unseen-project results in Table 2 is explained by measurable differences in task structure. The

CoDocBench-derived projects contain exactly one trace link per documentation item on average, corresponding to localized function-docstring-to-function matching. In contrast, the unseen projects require 5.53 trace links per documentation segment on average, and the TransArc systems require 15.77 links per documentation sentence on average. This structural difference is reflected in o3-mini’s recall: 98.2% on CoDocBench, compared with 52.6% on the unseen projects and 30.3% on TransArc. Thus, the stronger average recall in Table 5 is driven mainly by the more localized one-to-one structure of the CoDocBench portion of the Established Projects dataset, while TransArc illustrates that broader multi-link traceability remains substantially harder.

*2. The Precision-Recall Trade-off in Claude 4:* While Claude 4 Sonnet achieved high recall (95.1%), comparable to GPT-4o, its precision (60.4%) was notably lower, lagging behind GPT-4o by over 20 percentage points. In documentation-heavy projects (e.g., Pandas, Matplotlib), Claude tended to “over-retrieve” listing multiple plausible artifacts for a single description. This behavior mirrors the TF-IDF baseline, suggesting a retrieval strategy that favors completeness over selectivity, unlike the more precise OpenAI models.

*3. The TeaStore Anomaly:* TeaStore was the only project where TF-IDF (F1 48.6%) outperformed all LLMs (best LLM F1 40.2%). Appendix A.1 shows that TeaStore sits at the extreme end of our complexity spectrum: despite having the highest link density (21.2% of document–code pairs are true links), each documentation sentence must be linked to approximately 31 code files out of 145 candidates, yielding a combinatorial complexity of 93.2 bits under our index  $C_{avg}$ . Thus, the difficulty is not finding rare links in a sparse space, but correctly identifying a large subset of relevant artifacts among many plausible candidates. In this setting, TF-IDF’s broad keyword matching captured more of the required artifacts (Recall 65.3%) than the LLMs, which were too conservative and missed many true links (Recall 9–28%). This highlights a boundary condition: for extremely broad, low-context, high-combinatorial traceability tasks, statistical retrieval may still offer advantages over semantic reasoning.

### Key Findings for RQ1

**Traceability Performance.** All evaluated LLMs achieved high precision (87%–100%) with F1-scores between 64.2% and 80.4%, confirming their ability to create trace links; *Claude Sonnet 3.5* obtained the best F1 on both datasets.

**Common Error Patterns.** False positives stem mainly from (1) naming-based assumptions (**IAE**), (2) phantom links to non-existent artifacts (**PAL**), (3) over-generalised architectural patterns (**APB**), and (4) implementation overlinking to internal details (**IOL**).

**Documentation Cues Drive Recall.** In Crawl4AI (narrative-style documentation), *explicit mentions* of a code artifact are the strongest predictor of recall; in Unity Catalog (structured API documentation), the presence of structural cues, *Parameters / Returns* tables, plays the same role.

**Generalizability at Scale.** Experiments on 27 established projects confirm LLM dominance, with o3-mini achieving an average F1-score of 92.3%, significantly outperforming the best baseline (60.5%).

Table 5. Traceability Performance on Established Projects (25 CoDocBench projects + 2 TransArc systems). Comparison of Precision (P), Recall (R), and F1-score (F1) across LLMs and baselines averaged over five runs. The best F1 score per row is bolded.

Project	Claude 4 Sonnet			GPT-4o			o3-mini			TF-IDF	BM25	C-BERT
	P	R	F1	P	R	F1	P	R	F1	F1	F1	F1
AzurLaneAutoScript	0.635	0.984	0.771	0.854	0.985	0.915	0.967	1.000	<b>0.983</b>	0.558	0.213	0.016
StratosphereLinuxIPS	0.621	0.996	0.763	0.922	0.983	0.952	0.992	0.996	<b>0.994</b>	0.674	0.278	0.042
TTS	0.703	1.000	0.825	0.900	1.000	0.947	0.983	1.000	<b>0.991</b>	0.703	0.255	0.083
akshare	0.673	1.000	0.805	0.929	0.989	<b>0.958</b>	0.857	0.876	0.866	0.453	0.303	0.053
celery	0.581	1.000	0.735	0.810	1.000	0.895	0.892	1.000	<b>0.943</b>	0.826	0.202	0.087
certbot	0.759	0.993	0.860	0.993	1.000	0.996	1.000	1.000	<b>1.000</b>	0.926	0.378	0.072
cupy	0.822	1.000	0.902	0.988	1.000	0.994	1.000	1.000	<b>1.000</b>	0.705	0.125	0.042
diffusers	0.489	0.987	0.653	0.832	0.993	0.903	0.987	0.987	<b>0.987</b>	0.560	0.164	0.065
espnet	0.387	0.974	0.553	0.772	0.963	0.857	0.938	0.948	<b>0.943</b>	0.413	0.127	0.018
frappe	0.802	0.995	0.887	1.000	1.000	<b>1.000</b>	1.000	1.000	<b>1.000</b>	0.677	0.163	0.054
ipython	0.474	0.992	0.641	0.914	0.995	0.952	0.971	1.000	<b>0.985</b>	0.597	0.137	0.027
manim	0.820	1.000	0.900	0.935	0.993	0.963	0.973	1.000	<b>0.986</b>	0.714	0.206	0.067
matplotlib	0.309	0.965	0.468	0.737	0.993	0.846	0.874	0.923	<b>0.898</b>	0.545	0.058	0.007
mediastore	0.502	0.712	0.588	0.589	0.624	<b>0.606</b>	0.626	0.516	0.566	0.310	0.240	0.235
mindsdb	0.692	1.000	0.817	0.912	0.986	0.947	0.986	0.993	<b>0.990</b>	0.630	0.261	0.068
pandas	0.342	0.997	0.509	0.774	0.989	0.868	0.873	0.905	<b>0.889</b>	0.595	0.042	0.007
pyro	0.636	0.994	0.776	0.764	0.965	0.853	0.960	0.994	<b>0.977</b>	0.738	0.118	0.058
pytorch	0.511	0.967	0.669	0.808	0.984	0.887	0.965	0.981	<b>0.973</b>	0.643	0.055	0.013
pytorch_geometric	0.701	0.994	0.821	0.911	0.994	0.950	0.958	1.000	<b>0.979</b>	0.556	0.229	0.061
rasa	0.462	0.963	0.624	0.773	0.966	0.858	0.976	0.991	<b>0.983</b>	0.536	0.167	0.019
sentry	0.661	1.000	0.794	1.000	1.000	<b>1.000</b>	0.991	1.000	0.996	0.651	0.241	0.095
teastore	0.702	0.281	0.402	0.520	0.180	0.267	0.612	0.089	0.156	<b>0.486</b>	0.421	0.352
textual	0.844	0.993	0.912	0.966	1.000	0.982	0.993	1.000	<b>0.996</b>	0.618	0.239	0.069
tornado	0.472	0.996	0.640	0.680	0.996	0.808	0.898	1.000	<b>0.946</b>	0.565	0.094	0.040
transformers	0.440	0.909	0.593	0.591	0.991	0.740	0.969	0.969	<b>0.969</b>	0.471	0.100	0.031
zipline	0.603	1.000	0.751	0.855	0.982	0.914	0.954	1.000	<b>0.976</b>	0.630	0.109	0.043
zulip	0.658	0.988	0.789	0.866	0.996	0.926	0.904	0.996	<b>0.948</b>	0.553	0.138	0.040
AVERAGE	<b>0.604</b>	<b>0.951</b>	<b>0.720</b>	<b>0.837</b>	<b>0.946</b>	<b>0.881</b>	<b>0.930</b>	<b>0.932</b>	<b>0.923</b>	<b>0.605</b>	<b>0.187</b>	<b>0.065</b>

## 5 RQ2: TRACE LINK EXPLANATION

For RQ2, *How effectively can LLMs explain the nature of relationships between documentation and code elements?* we aim to analyze if LLMs can understand relationships between document and code elements. We present our motivation, approach, and results.

### 5.1 Motivation

In RQ1, we found that LLMs can accurately identify trace links between documentation segments and their corresponding code elements. While identifying trace links is fundamental, understanding the specific nature of these relationships is crucial for software maintenance and comprehension. Simple binary links (i.e., traceable vs non-traceable) do not capture how code implements, extends, or relates to documented functionality. We need to evaluate whether LLMs can provide meaningful explanations of these relationships that match human understanding. We limited this analysis to the manually constructed datasets because they include manually authored reference explanations describing the nature of each documentation-to-code relationship, whereas the Established Projects benchmark was derived primarily for retrieval-based trace-link evaluation rather than relationship-level explanation assessment.

## 5.2 Approach

To evaluate the selected LLMs' ability to explain relationships, we designed a two-part assessment process. First, we collected relationship explanations from each LLMs for all correctly identified trace links from RQ1. Then, due to the large volume of generated links, we developed an evaluation framework using a separate LLM as a judge [4, 94] to assess these explanations against our ground truth. The selected model for the LLM as a judge is Claude Sonnet 3.5 (claude-3-5-sonnet-20241022). The prompt for the evaluation can be found in the Appendix. To validate the reliability of using the LLM as a judge, we conducted a human validation study. The first author manually reviewed a random sample of 365 explanations (statistically determined for a 95% confidence level with a 5% margin of error). We calculated the agreement between the human validator and the LLM judge regarding the classification of explanations (Correct, Partially Correct, Incorrect). The resulting Cohen's Kappa was  $\kappa = 0.761$ , indicating substantial agreement [50]. This suggests that the LLM judge aligns reasonably well with human judgment, although some disagreement remains.

For each relationship evaluation, we provide the judge LLM with:

- The original documentation segment,
- The actual code artifact,
- The LLMs' generated relationship explanation,
- The ground truth relationship description.

We asked the judge LLM to classify the generated relationship explanation as correct (accurate and complete relationship description), partially correct (some aspects captured but others missed), and incorrect (failed to capture the true relationship). We deliberately focused only on True Positive cases from RQ1 for two key reasons. First, to evaluate an LLM's ability to explain relationships in a realistic scenario, it must first identify that a relationship exists. Second, using all possible document-code pairs returned by the LLM (i.e., including false positives) would involve asking the LLM to explain relationships where none exist as it is a false positive. In addition to the classification labels (correct, partially correct, and incorrect), we also asked the judge LLM to produce a justification for each classification and error types identified for partially correct and incorrect cases. In addition to the classification labels, the judge LLM was prompted to provide a rationale for its decision. The authors manually reviewed these rationales for all non-correct cases and consolidated them into high-level error categories (e.g., "Missing Key Details", "Unsupported Assumptions") using the same inductive coding approach described in RQ1. We report two complementary metrics: (1) *strict accuracy*, the percentage of explanations the judge classifies as *correct*, and (2) *relaxed accuracy*, the percentage classified as either *correct* or *partially correct*. Relaxed accuracy, therefore, shows how often the LLM captures the gist of the relationship even if some descriptive details are missing.

## 5.3 Results

Table 6 presents the results of our relationship explanations analysis across all three LLMs and both datasets. The results presented are the sum of explanations (TPs) across all runs. The results reveal interesting patterns in how different LLMs understand and explain documentation-code relationships.

**Relationship Explanation Performance.** All LLMs demonstrate strong capability in identifying the fundamental nature of relationships, with relaxed accuracy exceeding 97% across all LLM-dataset combinations. However, their ability to provide both complete and precise explanations (strict accuracy) varies with the majority below 58%. The o3-mini LLM shows stronger performance in strict accuracy compared to the other LLMs, achieving the highest scores of 54.8% and 71.1%

Table 6. Relationship Explanation Performance Across LLMs

LLM/Dataset	Total	Correct	Partial	Incorrect	Strict Acc.	Relaxed Acc.
<b>Claude Sonnet 3.5</b>						
Crawl4AI	2,200	983	1,196	21	44.7%	99.0%
Unity Catalog	580	334	245	1	57.6%	99.8%
<b>GPT-4o</b>						
Crawl4AI	1,687	723	927	37	42.9%	97.8%
Unity Catalog	412	200	211	1	48.5%	99.8%
<b>o3-mini</b>						
Crawl4AI	1,862	1,021	820	21	54.8%	98.9%
Unity Catalog	363	258	104	1	71.1%	99.7%

Table 7. Distribution of Error Types Across LLMs with Average Frequencies

Error Type	Claude Sonnet 3.5		GPT-4o		o3-mini		Avg. Freq. (%)
	Crawl4AI	Unity C.	Crawl4AI	Unity C.	Crawl4AI	Unity C.	
Missing Key Details	41.3%	42.3%	46.2%	37.7%	46.5%	21.9%	39.3%
Incomplete Explanation	50.2%	14.2%	42.3%	21.2%	39.0%	13.3%	30.0%
Implementation Detail	0.6%	10.2%	0.6%	10.4%	3.1%	25.7%	8.4%
Misinterpretation	1.6%	8.1%	1.4%	9.0%	1.3%	16.2%	6.3%
Unsupported Assumption	1.6%	15.9%	0.8%	9.0%	1.3%	3.8%	5.4%
Architectural	0.2%	6.1%	0.4%	7.1%	0.2%	12.4%	4.4%
Scope Mismatch	3.5%	0.4%	5.6%	0.9%	6.1%	1.9%	3.1%
Other	0.8%	2.8%	2.7%	4.7%	2.5%	4.8%	3.1%

Note: Unity C. is an abbreviation for Unity Catalog. Percentages are calculated relative to the total error counts for each LLM–dataset combination.

for Crawl4AI and Unity Catalog, respectively. This could perhaps be attributed to its reasoning capabilities.

A consistent pattern also emerges across all LLMs: explanation performance on Unity Catalog significantly exceeds that on Crawl4AI, with strict accuracy improvements ranging from 5.6 to 16.3 percentage points. This suggests that Unity Catalog’s more structured documentation style facilitates more complete and precise relationship explanations.

**Error Pattern Analysis.** Table 7 reports each error type as a percentage of the total non-correct explanations (i.e., partial or incorrect) for each LLM. The two “completeness” categories—*Missing Key Details* (average 39.3%) and *Incomplete Explanation* (average 30.0%)—dominate. We distinguish them as follows: *Missing Key Details* refers to explanations that identify the main relationship correctly but omit one or more important specifics or qualifiers (e.g., stating that `fetchData()` ‘retrieves results’ but failing to mention that the results are paginated), whereas *Incomplete Explanation* refers to explanations that convey only part of how a code artifact relates to the documentation.

*Implementation Detail* errors (average 8.4%) occur when explanations drift into low-level, internal specifics (e.g., implementation details of helper functions); such errors are more frequent in o3-mini, which could be attributed to its reasoning capabilities. *Misinterpretation* (average 6.3%) and *Unsupported Assumption* (average 5.4%) involve factually incorrect claims or unjustified inferences, such as overstating inheritance relationships. *Architectural* errors (4.4%) show that LLMs sometimes

misapply an observed pattern to unrelated parts of the system. *Scope Mismatch* (average 3.1%) errors arise when explanations are too broad or too narrow, whereas *Other* (average 3.1%) captures uncommon, idiosyncratic mistakes.

### Key Findings for RQ2

**Strong Essential Understanding.** LLMs demonstrate high capability in essential relationship identification, with relaxed accuracy consistently exceeding 97% across all LLMs and datasets.

**Weaker Complete Understanding.** In most cases, the LLMs have a strict of below 58% indicating a much weaker complete understanding.

**Error Characteristics.** Completeness-related errors dominate across all LLMs (average 69.3% of errors), with "Missing Key Details" and "Incomplete Explanation" being the primary issues rather than misinterpretation or incorrect understanding.

## 6 RQ3: INTERMEDIATE ELEMENT DETECTION

For RQ3, *How completely can LLMs identify intermediate elements in documentation-to-code trace chains?* we aim to analyze if LLMs can understand chains of connections between document and code elements. We present our motivation, approach, and results.

### 6.1 Motivation

Building on our earlier findings that LLMs can (i) identify direct document-to-code links (RQ1) and (ii) explain those links (RQ2), we now investigate whether they can also recover *multi-step chains* that include intermediate artifacts. Documentation often relates to code through chains of connections, where understanding intermediate elements is crucial for maintenance and comprehension. While identifying a single direct link is valuable (e.g., documentation  $\rightarrow$  class), real-world software often involves multiple code artifacts that together implement the documented feature. For instance, as discussed in Section 3.4, our *Crawl4AI* dataset includes a documentation segment about configuring the `max_concurrent_requests` parameter within the *Crawler* constructor, which in turn relies on `DEFAULT_CONCURRENCY` as a fallback. This scenario forms a trace chain of:

*Documentation*  $\rightarrow$  `Crawler.__init__`  $\rightarrow$  `Crawler.DEFAULT_CONCURRENCY`

Identifying the intermediate artifact reveals the underlying workflow. Therefore, we aim to determine whether LLMs can accurately detect these multi-step connections, which are critical for tasks such as change impact analysis in real-world software systems.

It is important to recognize that chain reconstruction is fundamentally more challenging than the independent link identification evaluated in RQ1. While RQ1 treats each potential link as an independent decision, chain reconstruction requires the model to correctly identify a sequence of dependent relationships, introducing unique failure modes such as error propagation and cumulative uncertainty. We analyze how these challenges manifest in our results in Section 7.3.

### 6.2 Approach

To evaluate the LLMs' ability to identify trace chains, we analyze chain completeness and accuracy. Using the same structured prompt as in RQ1, the model simultaneously (i) selects linked code artifacts from the full candidate set (RQ1), (ii) explains each predicted relationship in natural language (RQ2), and (iii) outputs a trace chain from the documentation segment to each selected artifact (RQ3); our RQ3 analysis then checks whether these trace chains correctly recover the

Table 8. Trace Chain Correctness Analysis Across LLMs

Dataset	Model	Total Chains	Complete Match (%)	Partial (Interm.) (%)	Partial (Prefix) (%)	Partial (Suffix) (%)	Incorrect (%)
<b>Crawl4AI</b>	Claude 3.5	2,200	23.0	67.8	8.3	0.0	0.0
	GPT-4o	1,687	30.6	68.0	1.4	0.1	0.0
	o3-mini	1,862	12.8	23.8	0.6	61.3	1.6
<b>Unity C.</b>	Claude 3.5	580	67.4	25.2	7.4	0.0	0.0
	GPT-4o	412	65.8	31.6	2.7	0.0	0.0
	o3-mini	363	80.2	6.1	1.4	12.4	0.0

Note: Unity C. is an abbreviation of Unity Catalog

intermediate elements between the documentation and the traced code element, without issuing any additional LLM calls.

For each generated chain, we analyze by parsing chains into their constituent elements and comparing them against ground truth chains. We evaluate trace chains using a structured classification system. Unlike the semantic explanations in RQ2, assessing chain correctness is an objective process. We compare the list of artifacts in the predicted chain against the ground truth chain to check for exact matches or structural deviations. Therefore, this analysis does not rely on subjective interpretation or LLM-based judging. The classification categories are as follows:

- Complete Match: Chain completely matches the ground truth,
- Partial Match (Intermediate): Correct endpoints but different middle elements,
- Partial Match (Prefix): Correctly starts with the first endpoint of the chain,
- Partial Match (Suffix): Correctly ends with the last endpoint of the chain,
- Incorrect: Completely different from ground truth.

Furthermore, for chains classified as *Partial Match (Intermediate)*, we algorithmically categorized the specific error pattern based on the structural difference relative to the ground truth. Chains containing extra intermediate nodes were labeled as **Extended**; chains missing required intermediate nodes were labeled as **Shortened**; and chains where an intermediate node was replaced by an incorrect one (preserving length) were labeled as **Substituted**.

### 6.3 Results

Table 8 categorizes the generated trace chains based on their alignment with the ground truth, classifying them as Complete Match, various types of Partial Match (Intermediate, Prefix, Suffix), or Incorrect. The chains considered are the sum across all runs. Two key patterns emerge:

*Errors Rarely Mean Complete Failure.* Across both datasets, outright incorrect chains are almost non-existent, except for o3-mini in the Crawl4AI dataset (1.6%). When LLMs fail to reproduce exact chains, they typically maintain meaningful connections by preserving either of the endpoints. This demonstrates that even partial traces often contain valuable relationship information.

*Chain Reconstruction Greatly Varies Across Datasets.* The most striking finding is the substantial difference in complete match rates between the datasets. All LLMs perform substantially better in *Unity Catalog* (structured API documentation) compared to *Crawl4AI* (narrative tutorial-style documentation), with Complete Match scores of 66-80% vs 13-31%. This suggests that formal, structured documentation might provide clearer signals for LLMs to accurately trace multi-step connections.

Table 9. Interior Error Pattern Distribution

Dataset	Model	Extended (%)	Shortened (%)	Substituted (%)
Crawl4AI	Claude 3.5	73.4	11.3	15.2
	GPT-4o	58.0	25.7	16.3
	o3-mini	58.0	15.8	26.2
Unity Catalog	Claude 3.5	92.5	0.0	7.5
	GPT-4o	41.5	43.1	15.4
	o3-mini	13.6	86.4	0.0

**Interior Error Patterns.** In Table 8, we observed that the cases where the intermediate nodes were incorrect (*Partial-Intermediate* chains) were the most common type of error. This is expected as we provide the LLM with the starting and end points in the prompt. Table 9 shows the three different patterns of intermediate errors:

- **Extended (extra nodes).** The most frequent pattern overall: 14-93% of intermediate chains have one or more additional artifacts, indicating that LLMs often “over-connect” the intended trace chain.
- **Shortened (missing nodes).** In 0-86% of cases, the LLM compresses the path by omitting at least one required step, suggesting an inclination to trim what it sees as redundant detail.
- **Substituted (node replaced).** When the error is neither pure addition nor pure omission, the LLM swaps an interior node for another, accounting for up to 26% of chains. Pure re-ordering without addition/removal is negligible (<0.1%) and is therefore omitted from the Table 9.

### Key Findings for RQ3

**LLMs do recover multi-step links, but completeness varies.** Exact matches are common on *Unity Catalog* (66-80%) yet scarce on *Crawl4AI* (13-31%), indicating that chain reconstruction greatly varies across datasets.

**When only the interior is wrong, three error patterns dominate.** Across all LLMs, interior deviations are mostly *Extended* paths (extra nodes, 14-93%), followed by *Shortened* paths (missing nodes, 0-86%), and *Substituted* paths (node swapped, up to 26%); pure re-ordering is negligible.

**Endpoints are reliable anchors.** Chains that lose both start and end points are below 2% for every LLM-dataset pair, so even imperfect traces give developers a sound entry and exit for navigation.

## 7 DISCUSSION

Our three research questions showed that LLMs (i) identify document-to-code links with high precision but uneven recall (RQ1); (ii) explain those links with good, though still partial, semantic fidelity (RQ2); and (iii) reconstruct multi-step chains once the link endpoints are clear (RQ3).

This section now turns from *what* the LLMs achieve to *how* task-framing influences those results, specifically, how context-window management (Section 7.1) and the amount of surrounding documentation (Section 7.2) help or hinder trace discovery. Claude Sonnet 3.5 serves as the running example because it was the strongest LLM overall; when GPT-4o or o3-mini behave differently under the same context settings, we highlight those differences in the text.



Table 10. Impact of Context Management Strategy on Traceability Performance

Dataset	Strategy	API Calls	TP	FP	FN	Prec.	Rec.	F1
Unity Catalog	One-to-many	32	116.0	17.4	39.0	0.870	0.748	0.804
	Many-to-many	1	5.0	5.2	150.0	0.496	0.032	0.061
Crawl4AI	One-to-many	112	440.0	22.0	207.0	0.953	0.680	0.794
	Many-to-many	1	17.6	3.2	624.4	0.829	0.027	0.053

Note: Results are averaged over five runs.

### 7.1 Impact of Context Management Strategies on LLM-Based Traceability

A critical challenge in leveraging LLMs for software traceability is how to effectively present documentation and code for analysis. This challenge stems from multiple constraints documented in the LLM literature. First, models have finite input context windows, and performance degrades as context length increases [51, 93]. Second, output token limits are typically more restrictive than input limits. Third, and perhaps most importantly for enumeration tasks, LLMs exhibit strong biases in list processing, most notably *enumeration bias*: they struggle to systematically list long sequences, tend toward small, “tidy” list sizes, and exhibit biased selection from long lists even when output capacity remains available [10, 15, 28]. Understanding how these constraints interact is crucial for designing effective LLM-based traceability tools.

There are three fundamental matching strategies for structuring traceability queries, each with distinct trade-offs:

**One-to-one matching strategy.** The most granular strategy examines each possible document-code pair individually. For instance, given two documentation segments ( $d_1, d_2$ ) and three code artifacts ( $c_1, c_2, c_3$ ), this strategy would make six separate queries:

$$\begin{aligned} d_1 &\leftrightarrow c_1, d_1 \leftrightarrow c_2, d_1 \leftrightarrow c_3 \\ d_2 &\leftrightarrow c_1, d_2 \leftrightarrow c_2, d_2 \leftrightarrow c_3 \end{aligned}$$

While this ensures focused attention on each potential relationship, it requires  $N \times M$  API calls for  $N$  documents and  $M$  code artifacts, making it prohibitively expensive for non-trivial projects.

**One-to-many matching strategy.** This strategy examines each documentation segment against all code artifacts simultaneously:

$$\begin{aligned} d_1 &\leftrightarrow \{c_1, c_2, c_3\} \\ d_2 &\leftrightarrow \{c_1, c_2, c_3\} \end{aligned}$$

This reduces API calls to  $N$  while maintaining focused analysis of each documentation segment. The LLM must identify which of the provided code artifacts trace to the current document.

**Many-to-many matching strategy.** The most computationally efficient strategy presents all documentation and code simultaneously in a single call:

$$\{d_1, d_2\} \leftrightarrow \{c_1, c_2, c_3\}$$

To empirically evaluate these trade-offs, we conducted a controlled experiment comparing our one-to-many strategy with a many-to-many strategy. We maintained identical conditions across both strategies, including five randomized runs, consistent prompting, and equal access to contextual information. Table 10 presents the results. We exclude one-to-one experiments due to prohibitive cost.

The results reveal a striking performance gap: across both datasets, many-to-many achieves recall below 3% and F1 around 5%, compared to approximately 80% F1 for one-to-many. Notably,

Table 11. Output Token Utilization: Many-to-Many vs. One-to-Many (Claude Sonnet 3.5)

Dataset	Strategy	Tokens/Response	Output Limit	Utilization
Crawl4AI	Many-to-many (1 call)	1,700–2,700	8,192	21–33%
	One-to-many (per call)	840–857	8,192	10%
	One-to-many (aggregate)	~95,000	—	—
Unity Catalog	Many-to-many (1 call)	1,300–1,700	8,192	16–21%
	One-to-many (per call)	786–818	8,192	10%
	One-to-many (aggregate)	~25,500	—	—

Note: Ranges reflect variation across five runs. Aggregate = total tokens across all calls.

precision remains moderate for many-to-many (49.6%–82.9%), indicating that the links returned are often valid—the model simply returns far too few of them. This pattern suggests a *recall* problem rather than a reasoning failure.

**Diagnosing the Many-to-Many Failure.** To understand why many-to-many fails so dramatically, we analyzed the actual token utilization of the LLM’s outputs. Claude Sonnet 3.5 has an output limit of 8,192 tokens. If the failure were due to hard output truncation, we would expect the model to consistently hit or approach this ceiling. Table 11 presents our token analysis.

The token analysis reveals a critical insight: *the model does not approach its output ceiling in many-to-many mode*. For Crawl4AI’s 112 documents, the model produces only 1,700–2,700 tokens total—approximately 25% of available capacity. This is far below what would be needed to enumerate all 645 ground-truth traces with explanations. In contrast, when processing one document at a time, the same model generates 840–857 tokens per response, aggregating to approximately 95,000 tokens across all calls.

This pattern is consistent with *enumeration bias* documented in recent LLM research [10, 15]. LLMs do not possess robust internal counters and struggle to systematically enumerate long sequences. They tend to terminate lists after a “reasonable-sounding” number of items, driven by training distributions dominated by short lists (“Top 5”, “Top 10”) and preference optimization that rewards concise, well-structured answers over exhaustive enumeration. In our setting, when asked to produce all trace links for all documents simultaneously, the model behaviorally stops after listing a modest number of mappings—not because it hits a hard limit, but because its learned priors favor compact outputs.

The one-to-many strategy succeeds precisely because it decomposes the task into  $N$  independent calls, each requiring enumeration of only a single document’s links. This keeps each enumeration task within the model’s “comfort zone” (typically 5–10 items per document) while the aggregate output across all calls achieves comprehensive coverage.

**Cost Models.** To help practitioners balance accuracy against computational cost, we provide cost models for each strategy:

$$C_{\text{one-to-one}} = N \times M \times t \times (i_d + i_a + s + o) \quad (1)$$

$$C_{\text{one-to-many}} = N \times t \times (i_d + M \times i_a + s + o) \quad (2)$$

$$C_{\text{many-to-many}} = t \times (N \times i_d + M \times i_a + s + o_{\max}) \quad (3)$$

Here,  $N$  represents the number of documentation segments,  $M$  the number of code artifacts, and  $t$  the cost per token. Input tokens are broken down into documentation ( $i_d$ ), artifact ( $i_a$ ), and

supporting context (*s*) components. The output tokens (*o*) are especially relevant: while many-to-many appears cheapest due to using only  $o_{\max}$  once, this theoretical savings is illusory because the strategy’s enumeration failure renders the output incomplete.

**Implications.** Our findings demonstrate that the many-to-many strategy’s apparent cost efficiency comes at the expense of catastrophic recall loss. The failure stems not from hard output truncation but from enumeration bias—a behavioral limitation where LLMs prefer compact, tidy outputs over exhaustive lists. For comprehensive traceability analysis, especially in projects with many documentation segments, the one-to-many strategy’s focused examination of each segment provides reliable and complete results. Practitioners should budget for *N* API calls (one per document) rather than attempting single-call solutions that current LLMs cannot execute reliably.

**Empirical Cost Analysis.** To provide concrete practical guidance, we report detailed cost metrics from our experiments using API pricing from early 2025. Our evaluation employed the one-to-many matching strategy with API caching enabled (as noted in Section 3.1).

Table 12 presents the token distribution and API call counts for our datasets. The token counts reflect the actual prompt structure: cached content (code artifacts and directory trees) remains constant across calls within a run and benefits from caching, while uncached content (documentation text) varies per call. For Crawl4AI (112 documents), we made 112 API calls with 1,934,099 total input tokens per run, comprising 22,931 uncached documentation tokens and 1,911,168 cached tokens. For Unity Catalog (32 documents), we made 32 API calls with 889,095 total input tokens per run, comprising 6,551 uncached and 882,544 cached tokens. For the larger dataset (27 Established Projects, 1,868 documents), we made 2,469 API calls with 78,542,580 total input tokens per run, comprising 372,831 uncached and 78,169,750 cached tokens. Output token counts varied by model based on response verbosity.

Table 12. Token Usage Distribution and API Calls Per Run

Dataset	Docs	API Calls	Input Tokens (Uncached / Cached)	Output Tokens (Approx.)
Crawl4AI	112	112	22,931 / 1,911,168	Claude 3.5: 9,286 GPT-4o: 6,956 o3-mini: 7,245
Unity Catalog	32	32	6,551 / 882,544	Claude 3.5: 2,651 GPT-4o: 1,985 o3-mini: 2,069
27 Established Projects	1,868	2,469	372,831 / 78,169,750	Claude 4: 117,294 GPT-4o: 70,442 o3-mini: 56,987

Table 13 presents the cost per run for each model and dataset. With caching enabled, the small dataset costs \$1.11-\$3.66 per run, while the large dataset costs \$26.33-\$99.35 per run. These translate to per-document costs of approximately 1.4¢ (Claude), 2.3¢ (o3-mini), and 5.3¢ (GPT-4o) when averaged across the datasets. The table also shows costs without caching, demonstrating the substantial impact of this optimization: caching reduces costs by 87-89% for Claude and approximately 50% for GPT-4o and o3-mini. This validates our selection criterion (Section 3.1) prioritizing models with caching support.

Table 13. Cost Per Run: With and Without Caching

Dataset	Model	With Caching	Without Caching
Crawl4AI (112 docs)	Claude 3.5	\$0.86	\$6.73
	GPT-4o	\$2.80	\$5.56
	o3-mini	\$1.23	\$2.45
Unity Catalog (32 docs)	Claude 3.5	\$0.25	\$1.92
	GPT-4o	\$0.80	\$1.59
	o3-mini	\$0.35	\$0.70
27 Established Projects (1,868 docs)	Claude 4	\$26.33	\$237.39
	GPT-4o	\$99.35	\$197.06
	o3-mini	\$43.65	\$86.65

Table 14. Performance Comparison: Segment-Only vs. File-Context Approaches

Dataset	Approach	TP	FP	FN	Prec.	Rec.	F1
Crawl4AI	Segment-Only	440.0	22.0	207.0	0.953	0.680	0.794
	File-Context	407.8	104.2	217.6	0.796	0.652	0.717
Unity Catalog	Segment-Only	116.0	17.4	39.0	0.870	0.748	0.804
	File-Context	127.4	23.4	27.0	0.845	0.825	0.835

Note: Results are averaged over five runs.

For scalability estimation, linear extrapolation suggests that tracing 10,000 documents would cost approximately \$140 (Claude), \$530 (GPT-4o), or \$230 (o3-mini) per complete repository trace. However, we note that documentation segment sizes vary across projects, so actual costs may differ. These represent one-time complete traces; incremental updates processing only modified documentation would be proportionally cheaper. Furthermore, because all API calls are independent and stateless, they can be executed in parallel, reducing wall-clock time to approximately that of a single API call rather than the sum of sequential calls.

7.2 Impact of Additional Contextual Information on Traceability Performance

In RQ1, for each documentation segment sent, we provide the LLM with the list of code artifacts and the directory tree structure of the repository. However, we hypothesize that more context might help the LLM identify additional relevant traces. Here, we explore the impact of a modified approach that provides the entire documentation file (from which a segment was derived) instead of just the documentation segment. To examine the impact of this additional contextual information on the LLM performance, Table 14 compares key metrics for the segment-only approach against the file-context approach.

As Table 14 shows, the file-context approach decreases precision and recall in Crawl4AI. Although it yields a slight increase in recall for Unity Catalog, it still results in more errors (FP + FN) overall. To investigate these outcomes further, we performed a qualitative analysis of false positives. Table 15 presents the error distributions for both datasets, revealing that Context Boundary Violation (CBV) (which was effectively absent for the segment-only approach) emerges prominently whenever the entire file is provided. In Crawl4AI, CBV constitutes 56.6% of all error tags under the file-context

Table 15. Error Analysis for Segment-Only vs. File-Context Approaches

Error Group	Crawl4AI		Unity Catalog	
	Segment (%)	File (%)	Segment (%)	File (%)
Implicit Assumption Errors (IAE)	61.3	26.3	51.9	34.0
Phantom Link (PAL)	30.7	2.0	15.5	17.7
Architecture Pattern Bias (APB)	0.0	2.0	32.6	12.8
Implementation Overlink (IOL)	8.0	13.2	0.0	1.4
Context Boundary Violation (CBV)	0.0	56.6	0.0	34.0
<i>Total Error</i>	150	152	129	141

approach, while in Unity Catalog it accounts for 34.0%. This suggests that when more context is included, the LLM often traces across segment boundaries and flags extraneous relationships.

These error distributions highlight the key difference introduced by file-level context: CBV becomes a major source of false positives, indicating that the LLM’s ability to focus on a specific segment is diluted by the surrounding text. Instead of confining traces within the relevant segment, the LLM “wanders” into other parts of the file, triggering incorrect links. So, while including the full documentation file might seem advantageous, it often produces confusion rather than clarity. The additional context allows the LLM to conflate content across segment boundaries, increasing false positives due to CBV. Therefore, practitioners should carefully consider the appropriate level of context when employing LLMs for traceability tasks.

### 7.3 The Challenge of Chain Reconstruction

As defined in Section 3.3, multi-step trace chains require identifying a sequence of dependent relationships, unlike independent links where each decision is isolated. Our results show that this sequential dependency introduces two distinct challenges that explain the performance gap between RQ1 and RQ3.

**Error Propagation.** In chain reconstruction, an error in identifying an early intermediate element can render the remainder of the chain unrecoverable. This is evident in our Crawl4AI results, where Partial Match (Prefix) errors (chains that correctly identify only the initial elements before diverging) account for up to 8.3% of cases (Table 8). Once the model makes an incorrect step, it cannot course-correct, as each subsequent prediction depends on the previous one. This contrasts sharply with independent link identification, where an error on one link does not affect the model’s ability to correctly identify others.

**Cumulative Uncertainty.** A second challenge is that confidence degrades with each step in the chain. While an LLM may exhibit high confidence in a single-step link (e.g., documentation → method), this confidence compounds negatively across multiple steps. Our results support this interpretation: all LLMs achieved substantially higher complete match rates on Unity Catalog (66–80%) compared to Crawl4AI (13–31%). Unity Catalog’s structured API documentation provides explicit, unambiguous dependency signals at each step, whereas Crawl4AI’s narrative tutorial style requires the model to infer implicit relationships, amplifying uncertainty at each link in the chain. The performance gap between datasets is far larger for chain reconstruction (RQ3) than for independent links (RQ1), suggesting that cumulative uncertainty disproportionately affects multi-step tasks.

## 8 IMPLICATIONS

The findings of our paper carry implications for software development & maintenance practices, for tooling & automation, and for AI/LLM research and development.

### 8.1 Software Development & Maintenance Practices

**Transforming Development Practices with LLM-Assisted Traceability.** Our results show that LLMs represent a significant advancement in automated traceability approaches. While existing automated methods like information retrieval (TF-IDF, BM25) and embedding-based techniques (CodeBERT) have helped reduce manual effort, they achieve F1-scores of only 36.2% to 69.3% in our experiments. LLMs, reaching up to 80.4%, offer a new level of accuracy that makes automated traceability more practical for daily development tasks. This improvement matters most in scenarios where traditional automated methods fall short: understanding implicit relationships, interpreting usage examples, and connecting high-level descriptions to specific implementations. For instance, when documentation describes a configuration parameter conceptually, LLMs can identify not just the parameter itself but also the methods that use it and the default values it falls back to. This semantic understanding transforms how teams approach documentation maintenance. During refactoring or API changes, developers can quickly assess the full impact across all documentation, not just places with exact keyword matches. New team members benefit particularly from this capability, as they can explore how concepts mentioned in tutorials connect to actual code implementations, building mental models faster than with traditional search-based approaches. The shift is from keyword-based discovery to semantic understanding, making documentation a more effective resource for both learning and maintenance.

**Integrating LLM Capabilities into Team Workflows.** Successfully incorporating LLMs into development practices requires thoughtful integration that respects both their strengths and limitations. Rather than treating LLMs as a wholesale replacement for existing processes, teams should view them as a new tier in their traceability toolkit. For routine documentation updates and initial impact analysis, LLMs can automate what previously required developer time. However, for critical documentation such as API contracts or security-related descriptions, human review remains essential. The key is establishing clear guidelines about when LLM suggestions can be trusted and when they require verification. Our findings suggest that explicit references and method-level traces are highly reliable, while class-level connections in narrative documentation need more scrutiny. Teams might establish workflows where LLM-generated links are automatically accepted for certain documentation types (like auto-generated API docs) but flagged for review in others (like architectural decision records). This stratified approach maximizes efficiency while maintaining quality. Additionally, teams should document their LLM usage patterns and share successful prompting strategies, building institutional knowledge about how to best leverage these tools. Over time, this creates a feedback loop where developers become better at working with LLMs, and the team develops customized approaches that fit their specific codebase and documentation style.

### 8.2 Tooling & Automation

**Design Principles for LLM-Enhanced Traceability Tools.** Our experiments show that architectural choices dramatically affect tool performance. The one-to-many matching strategy achieved F1-scores of 79.4%, while the many-to-many approach dropped to just 5.3%. This huge difference occurs because LLMs have strict output token limits and struggle to maintain focus when processing everything at once. Similarly, we found that providing entire documentation files actually hurt performance, increasing false positives by 56.6% as the LLM incorrectly linked content from outside

the relevant segment. This challenges the common assumption that more context always helps. Instead, tools need to break documentation into focused, manageable pieces that match what LLMs can effectively process. Based on these findings, future IDE plugins and documentation platforms should preprocess content into appropriate chunks, cache code artifact representations to reduce API costs, and track which documentation sections have been analyzed to prevent the LLM from making connections across segment boundaries. The key is working with LLM limitations rather than against them.

**Human-AI Collaboration in Traceability Tasks.** While LLMs rarely miss fundamental connections (with over 97% partial accuracy), they achieve full correctness only 42.9% to 71.1% of the time. This gap suggests that developers should view LLMs as assistants rather than replacements. The most effective approach combines LLM capabilities with human judgment: LLMs quickly identify potential links and explain basic relationships, while developers verify and refine these suggestions. This requires interfaces that go beyond simple accept/reject buttons. For example, when an LLM correctly identifies the start and end of a trace chain but gets the intermediate steps wrong (which happened in 13% to 87% of multi-step chains), developers need tools that let them easily fix the path while keeping the valid parts. Additionally, our results suggest combining LLMs with traditional static analysis. LLMs excel at understanding what documentation means and finding semantic connections, while static analysis can precisely trace execution paths. Tools that blend both approaches could offer developers the best of both worlds: semantic understanding from LLMs and technical precision from static analysis.

**Bridging Evaluation and Deployment.** Our experimental design employs pre-segmented documentation and defined artifact lists to enable rigorous, controlled evaluation. This raises an important question: how would these capabilities translate to real-world tools where such preprocessing may not exist? We emphasize the distinction between evaluation research (establishing whether a capability exists) and deployment engineering (building end-to-end systems). Our study addresses the former: demonstrating that LLMs possess the semantic reasoning capability to map documentation concepts to code implementations when given well-defined inputs. This represents a necessary foundation that if the capability does not exist under optimal conditions, it cannot exist under realistic conditions. Deployment would require addressing the preprocessing challenge, and the solution would vary by context. For projects with markdown-based documentation (as in our manually constructed datasets), segmentation could leverage existing structural markers (headings, code fences). For API documentation stored in structured formats (Javadoc, Sphinx), extraction could follow schema-defined boundaries. For legacy wikis or PDFs, more sophisticated document understanding techniques would be necessary. Similarly, code artifact enumeration could leverage language-specific tooling: Abstract Syntax Trees for parsing, IDE indexing services for project structure, or existing code search infrastructure. The structured prompts we used for evaluation would be encapsulated within tools, invisible to end-users. A developer would simply query “where is this feature implemented?” and the tool would internally construct appropriate inputs, invoke the LLM, and present results. Our findings establish the viability of the core traceability capability; translating it to diverse real-world contexts remains important future work.

**Optimizing for Large-Scale Deployment.** While our evaluation establishes the feasibility of LLM-based traceability, scaling to enterprise repositories with tens of thousands of artifacts would benefit from additional optimization strategies. Future work could explore several promising directions. First, *agentic systems with external memory* could enable learning across API calls, allowing the system to recognize patterns in previously traced relationships and avoid redundant analysis. Second, *search space reduction* through dependency analysis, architectural decomposition,

or impact analysis could narrow the candidate set before invoking the LLM. Third, integration with existing *retrieval-augmented generation (RAG)* techniques—such as LiSSA [34], which has demonstrated effectiveness in improving traceability link recovery while reducing search space—could combine the precision of semantic reasoning with the efficiency of targeted retrieval. Finally, *incremental tracing strategies* that focus only on changed documentation and affected code artifacts would substantially reduce costs for ongoing maintenance compared to complete re-analysis. Our evaluation demonstrates that the core semantic reasoning capability exists; these optimization strategies would translate that capability into practical, cost-effective deployment at industrial scale.

### 8.3 AI/LLM Research & Development

**Understanding LLM Limitations in Software Engineering.** Our analysis uncovered specific patterns in how LLMs fail at software tasks. The most common error type, implicit assumption errors (averaging 39.3% of all errors), shows that LLMs often confuse their general programming knowledge with the specific codebase they’re analyzing. They assume inheritance relationships based solely on class names, expect architectural patterns that do not exist, and create links to methods they think should be there but are not. These are not random mistakes but predictable biases that affect all three LLMs we tested. This suggests the problem lies in how current LLMs process information rather than just needing better training data. Researchers could address these issues by training models to explicitly recognize when not to make connections, developing architectures that better separate general knowledge from context-specific information, or creating prompting techniques that explicitly tell models to avoid assumptions.

**Building Better Evaluation Methods for LLMs in Software Engineering.** Our study provides both datasets and evaluation methods that future researchers can build on. By examining not just whether LLMs find links but also how well they explain them and trace multi-step connections, we showed that simple accuracy metrics hide important details. A model might correctly identify links but fail to explain them properly, making it less useful for developers who need to understand why connections exist. Our cost analysis, comparing different matching strategies and their API requirements, offers a practical framework for teams deciding how to deploy LLMs within budget constraints. The datasets we created, with 144 documentation segments and 105 code artifacts across two different documentation styles, test both obvious references and subtle implicit connections. Future work can extend this approach by testing on larger codebases, exploring whether specialized training improves performance, or developing standardized benchmarks for software engineering tasks. As LLMs become more common in development tools, we need rigorous evaluation methods that capture both their strengths and limitations in real-world scenarios.

## 9 THREATS TO VALIDITY

We discuss threats to the validity of our study across four dimensions: internal, external, construct, and conclusion validity.

### 9.1 Internal Validity

A key threat to internal validity arises from our ground-truth dataset creation. While we followed a systematic, step-by-step process for labeling and verifying trace links, there remains some inherent subjectivity when identifying implicit relationships between documentation and code. In particular, deciding whether a segment “indirectly references” a code element can vary between reviewers. We mitigated this by providing clear, explicit guidelines and holding iterative discussions in cases of disagreement.



Another threat is the non-deterministic nature of large language models (LLMs). Even with standardized prompts and carefully controlled parameters, repeated runs may yield variations in outputs. To mitigate this, we conducted *five runs* for each experiment and each LLM, using different random seeds and shuffled input orders, then aggregated the outcomes to minimize outlier effects.

Since we evaluated multiple LLMs, each with distinct pretraining corpora, subtle differences in prompt engineering could influence outcomes. Minor variations in how context is segmented or how instructions are phrased may alter performance. To reduce these risks, we applied a consistent prompting format (the one-to-many approach) across all LLMs, caching shared content (e.g., code blocks or file listings) so that each model received the same inputs.

Finally, for the Unseen Projects, we selected repositories created after each LLM’s training cutoff to reduce data contamination. There remains a small possibility that some patterns were still observed during pretraining (for example, through early forks or copied snippets), but we minimized this risk by verifying that these repositories are newer projects without obvious prior public forks. For the Established Projects (CoDocBench and TransArc), we make no such guarantee and instead treat them explicitly as “in-distribution” codebases that are likely present in the models’ pretraining corpora; these projects are used to study generalizability rather than performance on unseen training data.

## 9.2 External Validity

As detailed in Section 3.2, our evaluation spans 29 projects with varied documentation styles, but it still has important scope limitations.

First, regarding **programming language diversity**: our Unseen Projects span Python (Crawl4AI) and a mixed Java/Scala codebase (Unity Catalog), with trace links defined at class, method, and attribute levels. Our Established Projects combine 25 CoDocBench Python libraries with 2 Java microservice systems from TransArc (MediaStore and TeaStore); in this setting, CoDocBench contributes function-level docstrings and their corresponding Python functions, while TransArc contributes Java source files. We do not model cross-language links (for example, from Python front-ends to C++/CUDA backends in PyTorch or CuPy), so generalizing our findings to full multi-language traceability remains future work.

Second, regarding **documentation type scope**: our study focuses on *developer documentation* (API references, tutorials, and docstrings) as distinct from *requirements-level* artifacts such as architectural decision records, compliance documents, or formal specifications. These requirements-level artifacts represent a separate traceability problem addressed by prior work [5, 36, 39] and are beyond our current scope.

Third, regarding **legacy systems**: the evaluated projects are established and widely used but remain actively maintained. Legacy systems with severely outdated or inconsistent documentation may pose additional challenges not captured here.

Fourth, regarding **generalization to unstructured inputs**: our evaluation employs pre-defined documentation segments and code artifact lists to establish objective ground truth for measurement. This methodological choice, while necessary for valid evaluation (as discussed in Section 3.4), means our results reflect LLM performance when provided with well-structured inputs. Real-world deployment scenarios may involve documentation without clear structural boundaries (e.g., legacy wiki pages, scanned PDFs) or require dynamic discovery of relevant code artifacts without predefined candidate lists. In such contexts, overall system performance would depend on both the quality of automated preprocessing and the LLM’s semantic reasoning capability. Our evaluation isolates the latter, demonstrating that LLMs substantially outperform traditional baselines when both receive identical structured inputs. However, we cannot claim our measured performance levels (e.g., F1-scores of 64–80%) would transfer directly to scenarios where preprocessing quality

is imperfect or unavailable. Section 8.2 discusses potential approaches for bridging this gap in deployed tools.

Despite these boundaries, our 29-project evaluation represents a significant expansion over prior LLM traceability studies, which typically evaluate a smaller number of systems. Likewise, future work may compare an even broader set of LLMs, as emerging models continue to evolve rapidly.

### 9.3 Construct Validity

Construct validity threats involve how we define and operationalize “trace link”, “relationship explanation”, and “trace chain”. Our criteria distinguish between explicit references, implicit references, and multi-step connections. While these definitions are grounded in prior literature, other researchers might categorize or label relationships differently. We mitigated this through pilot testing on sample segments and ensuring our labeling guidelines were sufficiently detailed to capture both straightforward and subtle forms of traceability.

Our evaluation metrics—precision, recall, F1-scores for link detection, and correctness/completeness for relationship explanations—reflect commonly accepted measures in traceability. Yet, developers might additionally value metrics such as usability or domain relevance. We partially accounted for this by examining detailed explanation quality (e.g., completeness of reasoning) rather than just binary correctness. However, more nuanced or domain-specific constructs (e.g., compliance-level trace explanations) could require specialized measurement strategies.

A potential threat to construct validity in RQ2 arises from using an LLM to judge explanation quality for outputs generated by LLMs from the same architectural family. Although we used a different release of Claude Sonnet 3.5 as the judge (claude-3-5-sonnet-20241022) than the version being evaluated (claude-3-5-sonnet-20240620), models from the same family may share underlying biases that persist across releases. This could introduce circularity bias, where the judge systematically favors explanations that align with shared architectural priors. We considered using a judge from a different provider, but recent work on LLM-as-a-judge shows that large models across different providers share systematic evaluation biases such as position bias, verbosity bias, and self-preference bias [72, 82, 96]. Methodological guidance from this literature recommends validating LLM judges against task-specific human annotations rather than relying on model choice alone [11]. Following this guidance, we conducted a human validation study: the first author reviewed a statistically representative sample of 365 explanations (determined for a 95% confidence level with a 5% margin of error), yielding substantial agreement ( $\kappa = 0.761$ ) with the LLM judge. However, this agreement is not perfect, so the RQ2 results should be interpreted as approximate evidence of explanation quality at the aggregate level rather than error-free ground truth for every individual explanation.

### 9.4 Conclusion Validity

One potential threat to conclusion validity is the inherent variability of LLM-based systems. Providers routinely update models, and performance on identical prompts may drift over time. We mitigated this by recording the exact version identifier for each LLM used in our experiments, carrying out all runs within a defined time window. This ensures that subsequent researchers could reference the same model versions should they wish to replicate our procedures.

We further strengthened our conclusions through repeated experiments (five runs per LLM for all experiments) and standardized prompt structures, thus capturing a more robust view of performance. These measures reduce the likelihood that any single run’s outlier behavior skews the findings. While future model updates or additional hyperparameter tuning might alter results, our detailed documentation of methods and model versions supports replicability.

Lastly, we acknowledge that the scope of our findings is bounded by the specific LLMs, prompt structures, and projects we evaluated. Although we selected *new* open-source repositories and performed a comprehensive analysis of documentation-to-code links, different codebases or LLM architectures might reveal additional nuances. Our work nevertheless provides a reproducible foundation for subsequent studies to expand upon or refine.

## 10 CONCLUSION

This work set out to systematically evaluate how LLMs perform on documentation-to-code traceability tasks, using two newer open-source projects specifically chosen to reduce the risk of data contamination from the LLMs’ pretraining corpora. Overall, our results indicate that LLMs substantially outperform traditional information-retrieval baselines when identifying direct trace links, yet offer only partial completeness in their explanatory details. In multi-step or chain-based relationships, the LLMs consistently achieve high endpoint accuracy but sometimes struggle with precisely identifying intermediate elements. A closer look at these results reveals three themes. First, LLM-based trace discovery exhibits decent performance, providing clear evidence of their ability to interpret documentation. Second, while LLMs seldom fail entirely on explanation tasks—often capturing the core relationship—they remain prone to omissions of more subtle yet potentially important nuances.

These results indicate that LLMs have strong potential for significantly reducing the manual effort required to create trace links, especially when documentation follows coherent, well-structured conventions that make it easier for a language model to associate relevant code artifacts. At the same time, developers should remain aware that LLM-generated explanations may omit subtle but important information. Our findings thus point to a collaboration model where LLMs provide an efficient “first pass” at trace creation and refinement, while human reviewers handle complex or domain-specific nuances. In practice, this suggests integrating simple accept/reject controls for LLM-suggested links into IDEs or documentation portals, so teams gain speed without sacrificing accuracy.

## REFERENCES

- [1] Bassem Abid, Jane Cleland-Huang, and Ying Zou. 2014. Using traceability links to recommend adaptive changes for documentation evolution. *IEEE Transactions on Software Engineering* 40, 10 (2014), 997–1013.
- [2] Emad Aghajani, Csaba Nagy, Mario Linares-Vásquez, Laura Moreno, Gabriele Bavota, Michele Lanza, and David C Shepherd. 2020. Software documentation: the practitioners’ perspective. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 590–601.
- [3] Emad Aghajani, Csaba Nagy, Olga Lucero Vega-Márquez, Mario Linares-Vásquez, Laura Moreno, Gabriele Bavota, and Michele Lanza. 2019. Software documentation issues unveiled. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 1199–1210.
- [4] Toufique Ahmed, Premkumar Devanbu, Christoph Treude, and Michael Pradel. 2024. Can LLMs replace manual annotation of software engineering artifacts? *arXiv preprint arXiv:2408.05534* (2024).
- [5] Syed Juned Ali, Varun Naganathan, and Dominik Bork. 2024. Establishing Traceability Between Natural Language Requirements and Software Artifacts by Combining RAG and LLMs. In *International Conference on Conceptual Modeling*. Springer, 295–314.
- [6] Nouf Alturayef, Jameleddine Hassine, and Irfan Ahmad. 2025. Machine learning approaches for automated software traceability: A systematic literature review. *Journal of Systems and Software* 230 (2025), 112536.
- [7] Anthropic. [n. d.]. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>
- [8] AI Anthropic. 2024. Claude 3.5 sonnet model card addendum. *Claude-3.5 Model Card* 3, 6 (2024).
- [9] Giuliano Antoniol, Gerardo Canfora, Gerardo Casazza, Andrea De Lucia, and Ettore Merlo. 2002. Recovering traceability links between code and documentation. *IEEE transactions on software engineering* 28, 10 (2002), 970–983.
- [10] Thomas Ball, Shuo Chen, and Cormac Herley. 2024. Can we count on llms? the fixed-effect fallacy and claims of gpt-4 capabilities. *arXiv preprint arXiv:2409.07638* (2024).

- [11] Anna Bavaresco, Raffaella Bernardi, Leonardo Bertolazzi, Desmond Elliott, Raquel Fernández, Albert Gatt, Esam Ghaleb, Mario Giulianelli, Michael Hanna, et al. 2025. LLMs instead of Human Judges? A Large Scale Empirical Study across 20 NLP Evaluation Tasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*.
- [12] Daniel Blasco, Carlos Cetina, and Óscar Pastor. 2020. A fine-grained requirement traceability evolutionary algorithm: Kromaia, a commercial video game case study. *Information and Software Technology* 119 (2020), 106235.
- [13] Markus Borg, Krzysztof Wnuk, and Dietmar Pfahl. 2012. Industrial comparability of student artifacts in traceability recovery research-an exploratory survey. In *2012 16th European Conference on Software Maintenance and Reengineering*. IEEE, 181–190.
- [14] Héctor Cadavid, Vasilios Andrikopoulos, and Paris Avgeriou. 2022. Documentation-as-code for interface control document management in systems of systems: A technical action research study. In *European Conference on Software Architecture*. Springer, 19–37.
- [15] Zhaoyang Cao, Lael Schooler, and Reza Zafarani. 2025. Analyzing Memory Effects in Large Language Models through the lens of Cognitive Psychology. *arXiv preprint arXiv:2509.17138* (2025).
- [16] Sandra Casas, Diana Cruz, Graciela Vidal, and Marcela Constanzo. 2021. Uses and applications of the OpenAPI/Swagger specification: a systematic mapping of the literature. In *2021 40th International Conference of the Chilean Computer Science Society (SCCC)*. IEEE, 1–8.
- [17] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [18] Xiaofan Chen and John Grundy. 2011. Improving automated documentation to code traceability by combining retrieval techniques. In *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. IEEE, 223–232.
- [19] Adelina Ciurumelea, Carol V Alexandru, Harald C Gall, and Sebastian Proksch. 2023. Completing Function Documentation Comments Using Structural Information. *Empirical Software Engineering* 28, 4 (2023), 86.
- [20] Jane Cleland-Huang. 2011. Traceability in agile projects. In *Software and Systems Traceability*. Springer, 265–275.
- [21] Jane Cleland-Huang, Adam Czauderna, Marek Gibiec, and John Emenecker. 2010. A machine learning approach for tracing regulatory codes to product specific requirements. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering—Volume 1*. 155–164.
- [22] Jane Cleland-Huang, Orlena Gotel, Andrea Zisman, et al. 2012. *Software and systems traceability*. Vol. 2. Springer.
- [23] Jane Cleland-Huang, Orlena CZ Gotel, Jane Huffman Hayes, Patrick Mäder, and Andrea Zisman. 2014. Software traceability: trends and future directions. In *Future of software engineering proceedings*. 55–69.
- [24] Andrea De Lucia, Fausto Fasano, Rocco Oliveto, and Genoveffa Tortora. 2006. Can information retrieval techniques effectively support traceability link recovery?. In *14Th IEEE international conference on program comprehension (ICPC'06)*. IEEE, 307–316.
- [25] Google Deepmind. [n. d.]. Google Gemini 2.0 Pro. <https://deepmind.google/technologies/gemini/pro/>
- [26] Wei Ding, Peng Liang, Antony Tang, and Hans Van Vliet. 2014. Knowledge-based approaches in software documentation: A systematic literature review. *Information and Software Technology* 56, 6 (2014), 545–567.
- [27] Shubhang Shekhar Dvivedi, Vyshnav Vijay, Sai Leela Rahul Pujari, Shoumik Lodh, and Dhruv Kumar. 2024. A comparative analysis of large language models for code documentation generation. In *Proceedings of the 1st ACM International Conference on AI-Powered Software*. 65–73.
- [28] Jonathan E Eicher and RF Irgolić. 2024. Reducing Selection Bias in Large Language Models. *arXiv preprint arXiv:2402.01740* (2024).
- [29] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. 2023. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 31–53.
- [30] Leye Feng, Lin Ma, and Xin Zhang. 2024. Recovering Traceability Links between Release Notes and Related Software Artifacts. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. IEEE, 1–4.
- [31] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).
- [32] Andrew Forward and Timothy C Lethbridge. 2002. The relevance of software documentation, tools and technologies: a survey. In *Proceedings of the 2002 ACM symposium on Document engineering*. 26–33.
- [33] Dominik Fuchß, Sophie Corallo, Jan Keim, Janek Speit, and Anne Koziolk. 2022. Establishing a benchmark dataset for traceability link recovery between software architecture documentation and models. In *European Conference on Software Architecture*. Springer, 455–464.

- [34] Dominik Fuchß, Tobias Hey, Jan Keim, Haoyu Liu, Niklas Ewald, Tobias Thirof, and Anne Koziulek. 2025. LiSSA: toward generic traceability link recovery through retrieval-augmented generation. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering. ICSE*, Vol. 25.
- [35] Dominik Fuchß, Haoyu Liu, Tobias Hey, Jan Keim, and Anne Koziulek. 2025. Enabling architecture traceability by llm-based architecture component name extraction. In *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*. IEEE, 1–12.
- [36] Chuyan Ge, Tiantian Wang, Xiaotian Yang, and Christoph Treude. 2025. Cross-Level Requirements Tracing Based on Large Language Models. *IEEE Transactions on Software Engineering* 51, 7 (2025), 2044–2066.
- [37] Orlena Gotel, Jane Cleland-Huang, Jane Huffman Hayes, Andrea Zisman, Alexander Egyed, Paul Grünbacher, Alex Dekhtyar, Giuliano Antoniol, and Jonathan Maletic. 2012. The grand challenge of traceability (v1. 0). *Software and systems traceability* (2012), 343–409.
- [38] Jin Guo, Jinghui Cheng, and Jane Cleland-Huang. 2017. Semantically enhanced software traceability using deep learning techniques. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 3–14.
- [39] Jameleddine Hassine. 2024. An llm-based approach to recover traceability links between security requirements and goal models. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 643–651.
- [40] Jane Huffman Hayes and Alex Dekhtyar. 2005. A framework for comparing requirements tracing experiments. *International Journal of Software Engineering and Knowledge Engineering* 15, 05 (2005), 751–781.
- [41] Jane Huffman Hayes, Alex Dekhtyar, and James Osborne. 2003. Improving requirements tracing via information retrieval. In *Proceedings. 11th IEEE International Requirements Engineering Conference, 2003*. IEEE, 138–147.
- [42] Jane Huffman Hayes, Alex Dekhtyar, and Senthil Karthikeyan Sundaram. 2006. Advancing candidate link generation for requirements tracing: The study of methods. *IEEE Transactions on Software Engineering* 32, 1 (2006), 4–19.
- [43] Vibhashree Hippargi, Erik Kamsties, and Jürgen Naumann. 2025. Evaluating the Capabilities of LLMs in Traceability Maintenance for Automotive System and Software Requirements. In *Proceedings of the 31st International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ 2025)*.
- [44] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.
- [45] Hao Hu, Haiyang Wang, and Qing Wang. 2017. Refining Traceability Links between Code and Software Documents. In *Proceedings of the 9th Asia-Pacific Symposium on Internetware*. ACM, 1–9.
- [46] Hao Hu, Haiyang Wang, and Qing Wang. 2018. Toward accurate link between code and software documentation. *Science China Information Sciences* 61, 1 (2018), 1–3.
- [47] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- [48] Nikitas Karanikolas, Eirini Manga, Nikolettta Samaridi, Eleni Tousidou, and Michael Vassilakopoulos. 2023. Large language models versus natural language understanding and generation. In *Proceedings of the 27th Pan-Hellenic Conference on Progress in Computing and Informatics*. 278–290.
- [49] Junaed Younus Khan and Gias Uddin. 2022. Automatic code documentation generation using gpt-3. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–6.
- [50] J Richard Landis and Gary G Koch. 1977. The measurement of observer agreement for categorical data. *biometrics* (1977), 159–174.
- [51] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173.
- [52] Andrea De Lucia, Fausto Fasano, Rocco Oliveto, and Genoveffa Tortora. 2007. Recovering traceability links in software artifact management systems using information retrieval methods. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 16, 4 (2007), 13–es.
- [53] Yuzhan Ma, Sarah Fakhoury, Michael Christensen, Venera Arnaoudova, Waleed Zogaan, and Mehdi Mirakhorli. 2018. Automatic classification of software artifacts in open-source applications. In *Proceedings of the 15th International Conference on Mining Software Repositories*. 414–425.
- [54] Walid Maalej and Martin P Robillard. 2013. Patterns of knowledge in API reference documentation. *IEEE Transactions on software Engineering* 39, 9 (2013), 1264–1282.
- [55] Ana C Marcén, Raúl Lapeña, Oscar Pastor, and Carlos Cetina. 2020. Traceability link recovery between requirements and models using an evolutionary algorithm guided by a learning to rank algorithm: Train control and management case. *Journal of Systems and Software* 163 (2020), 110519.
- [56] Andrian Marcus and Jonathan I Maletic. 2003. Recovering documentation-to-source-code traceability links using latent semantic indexing. In *25th International Conference on Software Engineering, 2003. Proceedings*. IEEE, 125–135.

- [57] Arsalan Masoudifard, Mohammad Mowlavi Sorond, Moein Madadi, Mohammad Sabokrou, and Elahe Habibi. 2024. Leveraging Graph-RAG and Prompt Engineering to Enhance LLM-Based Automated Requirement Traceability and Compliance Checks. *arXiv preprint arXiv:2412.08593* (2024).
- [58] Wail S. Masri, Maroun M. Diab, and Elia Abu Shama. 2018. Mining and discovery of hidden relationships between software source codes and related textual documents. *Digital Scholarship in the Humanities* 33, 2 (2018), 261–279.
- [59] Chris Mills. 2017. Automating traceability link recovery through classification. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. 1068–1070.
- [60] Chris Mills, Javier Escobar-Avila, and Sonia Haiduc. 2018. Automatic traceability maintenance via machine learning classification. In *2018 IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 369–380.
- [61] Laura Moreno, Andrian Marcus, Lori Pollock, and K Vijay-Shanker. 2013. Jsummarizer: An automatic generator of natural language summaries for java classes. In *2013 21st International Conference on Program Comprehension (ICPC)*. IEEE, 230–232.
- [62] Nachiappan Nagappan, Brendan Murphy, and Victor Basili. 2008. The influence of organizational structure on software quality: an empirical case study. In *Proceedings of the 30th international conference on Software engineering*. 521–530.
- [63] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an llm to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [64] Marc North, Amir Atapour-Abarghouei, and Nelly Bencomo. 2024. Code gradients: Towards automated traceability of llm-generated code. In *2024 IEEE 32nd International Requirements Engineering Conference (RE)*. IEEE, 321–329.
- [65] OpenAI. [n. d.]. <https://platform.openai.com>
- [66] OpenAI. 2025. Introducing GPT-4.1 in the API | openai. <https://openai.com/index/gpt-4-1/>
- [67] OpenAI. 2025. Models. <https://platform.openai.com/docs/models/gpt-4o> [Accessed 04-03-2025].
- [68] OpenAI. 2025. Models. <https://platform.openai.com/docs/models/o3-mini> [Accessed 04-03-2025].
- [69] OpenAI. 2025. OpenAI o3-mini. <https://openai.com/index/openai-o3-mini> [Accessed 04-03-2025].
- [70] Ipek Ozkaya. 2023. Application of large language models to software engineering tasks: Opportunities, risks, and implications. *IEEE Software* 40, 3 (2023), 4–8.
- [71] Kunal Pai, Premkumar Devanbu, and Toufique Ahmed. 2024. *CoDocBench: A Dataset for Code-Documentation Alignment in Software Maintenance*. <https://doi.org/10.5281/zenodo.14251622>
- [72] Arjun Panickssery, Samuel R Bowman, and Shi Feng. 2024. LLM Evaluators Recognize and Favor Their Own Generations. In *Advances in Neural Information Processing Systems*.
- [73] David Lorge Parnas. 2010. Precise documentation: The key to better software. In *The future of software engineering*. Springer, 125–148.
- [74] Rolf-Helge Pfeiffer. 2020. What constitutes software? An empirical, descriptive study of artifacts. In *Proceedings of the 17th International Conference on Mining Software Repositories*. 481–491.
- [75] Marco Raglianti. 2022. Topology of the documentation landscape. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 297–299.
- [76] Marco Raglianti, Csaba Nagy, Roberto Minelli, Bin Lin, and Michele Lanza. 2023. On the rise of modern software documentation (pearl/brave new idea). In *37th European Conference on Object-Oriented Programming (ECOOP 2023)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 43–1.
- [77] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.
- [78] Alberto D. Rodriguez, Katherine R. Dearstyne, and Jane Cleland-Huang. 2023. Prompts Matter: Insights and Strategies for Prompt Engineering in Automated Software Traceability. In *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*. IEEE, 455–464.
- [79] Marcela Ruiz, Jin Yang Hu, and Fabiano Dalpiaz. 2023. Why don't we trace? A study on the barriers to software traceability in practice. *Requirements Engineering* 28, 4 (2023), 619–637.
- [80] SEART. 2025. GitHub Search. <https://seart-ghs.si.usi.ch/> [Accessed 05-03-2025].
- [81] SEART. 2025. GitHub Search. <https://github.com/seart-group/ghs> [Accessed 05-03-2025].
- [82] Lin Shi, Chiyu Ma, Wenhua Liang, Xingjian Diao, Weicheng Ma, and Soroush Vosoughi. 2025. Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge. *arXiv preprint arXiv:2406.07791* (2025).
- [83] Lakmal Silva, Michael Unterkalmsteiner, and Krzysztof Wnuk. 2023. Towards identifying and minimizing customer-facing documentation debt. In *2023 ACM/IEEE International Conference on Technical Debt (TechDebt)*. IEEE, 72–81.
- [84] Daniela Steidl, Benjamin Hummel, and Elmar Juergens. 2013. Quality analysis of source code comments. In *2013 21st international conference on program comprehension (icpc)*. Ieee, 83–92.
- [85] Wen Siang Tan, Markus Wagner, and Christoph Treude. 2024. Detecting outdated code element references in software repository documentation. *Empirical Software Engineering* 29, 1 (2024), 5.

- [86] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2017. An Empirical Comparison of Model Validation Techniques for Defect Prediction Models. 1 (2017).
- [87] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2018. The Impact of Automated Parameter Optimization for Defect Prediction Models. (2018).
- [88] Fangchao Tian, Tianlu Wang, Peng Liang, Chong Wang, Arif Ali Khan, and Muhammad Ali Babar. 2021. The impact of traceability on software maintenance and evolution: A mapping study. *Journal of Software: Evolution and Process* 33, 10 (2021), e2374.
- [89] Gias Uddin and Martin P Robillard. 2015. How API documentation fails. *Ieee software* 32, 4 (2015), 68–75.
- [90] Esther Van der Linde, Daniel Kless, and Gustaf Neumann. 2020. Text Similarity Between Concepts Extracted from Source Code and Documentation. In *Advances in Artificial Intelligence*. Springer, 72–85.
- [91] Esther van der Linde, Frederik Schaefer, and Daniel Kless. 2021. Extracting and Comparing Concepts Emerging from Software Code, Documentation and Tests. In *Proceedings of the 2021 2nd International Workshop on AI in Requirements Engineering*. ACM, 1–8.
- [92] Alejandro Velasco and Jairo Aponte. 2020. Automated fine grained traceability links recovery between high level requirements and source code implementations. *ParadigmPlus* 1, 2 (2020), 18–41.
- [93] Minzheng Wang, Longze Chen, Cheng Fu, Shengyi Liao, Xinghua Zhang, Bingli Wu, Haiyang Yu, Nan Xu, Lei Zhang, Run Luo, et al. 2024. Leave no document behind: Benchmarking long-context llms with extended multi-doc qa. *arXiv preprint arXiv:2406.17419* (2024).
- [94] Ruiqi Wang, Jiyu Guo, Cuiyun Gao, Guodong Fan, Chun Yong Chong, and Xin Xia. 2025. Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering. *arXiv preprint arXiv:2502.06193* (2025).
- [95] Zezhou Yang, Sirong Chen, Cuiyun Gao, Zhenhao Li, Xing Hu, Kui Liu, and Xin Xia. 2025. An Empirical Study of Retrieval-Augmented Code Generation: Challenges and Opportunities. *ACM Transactions on Software Engineering and Methodology* (2025).
- [96] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*.
- [97] Zibin Zheng, Kaiwen Ning, Qingyuan Zhong, Jiachi Chen, Wenqing Chen, Lianghong Guo, Weicheng Wang, and Yanlin Wang. 2025. Towards an understanding of large language models in software engineering tasks. *Empirical Software Engineering* 30, 2 (2025), 50.
- [98] Hucheng Zhou, Jian-Guang Lou, Hongyu Zhang, Haibo Lin, Haoxiang Lin, and Tingting Qin. 2015. An empirical study on quality issues of production big data platform. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 2. IEEE, 17–26.
- [99] Shuyan Zhou, Uri Alon, Frank F Xu, Zhiruo Wang, Zhengbao Jiang, and Graham Neubig. 2022. Docprompting: Generating code by retrieving the docs. *arXiv preprint arXiv:2207.05987* (2022).
- [100] Thomas Zimmermann and Peter Weißgerber. 2004. Preprocessing CVS Data for Fine-Grained Analysis.. In *MSR*, Vol. 4. 2–6.

## A APPENDIX

Below we provide additional details of our methodology. Figure 5 in the main body illustrates our prompt structure for documentation-to-code traceability. Here, we also include the code for asynchronous document processing used in our experiments.

Listing 1. Full Evaluation Prompt

```

async def process_document(idx: int, request: Dict, total: int,
 system_message: str, results_dir: str) -> List[Dict]:
 """
 Process a single document request asynchronously.
 Prints status messages, builds the prompt, calls the LLM concurrently
 ,
 and processes the response.
 """
 print(f"\nProcessing document {idx}/{total}")

```

```

Prepare cached content with all available artifacts and optional
 document context
cached_content = {
 "available_artifacts": [
 {
 "artifact_id": artifact["artifact_id"],
 "title": artifact["title"],
 "location": artifact["location"],
 "content": artifact["content"]
 }
 for artifact in request["artifact_list"]
],
 "directory_tree": request["directory_tree"]
}

if request.get("document_context"):
 cached_content["document_context"] = request["document_context"]

cached_message = json.dumps(cached_content, indent=2)

Construct prompt
prompt_dict = {
 "task": "Documentation to Code Traceability Analysis",
 "description": "Analyze the documentation snippet and identify
 code artifacts that implement or relate to the documented
 functionality. Use only artifact names from the provided list
 .",
 "data": {
 "document": {
 "text": {
 "value": request["doc_text"],
 "description": "Documentation text snippet to analyze
 for trace links"
 },
 "location": {
 "value": request["doc_location"],
 "description": "Location information for context"
 },
 "file": {
 "value": "", # request["document_file"] omitted to
 run without file
 "description": "File from which the document.text
 snippet was extracted, for context, the file may
 or may not be included"
 }
 }
 },
 "instructions": {

```



```

"steps": [
 "1. Focus ONLY on the provided 'text' field when
 identifying traces",
 "2. The location and file information are provided for
 context only - DO NOT trace to elements mentioned
 elsewhere in the file",
 "3. Analyze the specific text snippet to identify
 explicitly mentioned code elements",
 "4. Identify any code elements demonstrated in usage
 examples within this text",
 "5. For each identified artifact:",
 " - If it's explicitly mentioned in the text snippet,
 mark as 'explicit'",
 " - If it appears in usage examples within this text
 but isn't directly discussed, mark as 'implicit'",
 " - Explain the specific nature of how it relates to
 the documentation",
 " - Identify if it's part of a chain/pathway to other
 artifacts",
 "6. Only trace to:",
 " - Classes and their usage",
 " - Methods and their usage",
 " - Class-level attributes that form part of the public
 interface and their usage",
 "7. DO NOT trace to:",
 " - Individual statements within methods",
 " - Implementation details",
 " - Local variables or parameters",
 " - Elements mentioned outside the provided text
 snippet",
 "8. For each relationship:",
 " - Provide specific evidence from the text",
 " - Explain the nature of the relationship in detail",
 " - Identify any intermediate steps or dependencies",
 "9. Make sure to include all required fields in your
 response",
 "10. Ensure you only trace to the included list of code
 snippets returning only titles within that list given
 ",
 "11. For any traced class, also trace to its base classes
 and implementing classes if they are in the
 available artifacts list",
 "12. For traceability pathways:",
 " - Always start with the document name (extracted from
 location)",
 " - Use exact artifact titles for code elements",
 " - Strictly follow 'A -> B -> C' format with spaces
 around arrows",

```

```

 " - Must end at the traced artifact",
 " - Explain why each intermediate step is necessary"
],
 "output_format": {
 "artifacts": [
 {
 "artifact_id": {
 "type": "integer",
 "description": "Unique identifier for the
 artifact"
 },
 "title": {
 "type": "string",
 "description": "Exact artifact title from
 available_artifacts"
 },
 "relationship": {
 "type": "string",
 "enum": ["explicit", "implicit"],
 "description": "Whether artifact is
 explicitly mentioned or implicitly used"
 },
 "relationship_type": {
 "type": "string",
 "description": "Nature of relationship (
 implements, extends, uses, etc.)"
 },
 "relationship_explanation": {
 "type": "string",
 "description": "Detailed explanation with
 evidence from text"
 },
 "trace_chain": {
 "type": "string",
 "description": "Format: doc_name.md ->
 Artifact1 -> Artifact2"
 },
 "trace_chain_explanation": {
 "type": "string",
 "description": "Explanation of chain
 relationships"
 }
 }
]
 }
}

```

Listing 2. Prompt for LLM as a Judge in RQ2

```
prompt = {
 "task": "Judge Relationship Description (3-category)",
 "context": {
 "code": code_content,
 "documentation": doc_text
 },
 "descriptions": {
 "predicted": predicted,
 "ground_truth": ground_truth
 },
 "instructions": ""
 You are an expert judge comparing two relationship
 descriptions:
 (1) predicted vs. (2) ground_truth.

 Focus on whether the predicted description gets the high-
 level
 relationship right. Do NOT penalize minor omissions
 about code details or parameters. Only label as
 partially_correct if
 the predicted text contradicts or significantly
 misunderstands a
 major aspect of the ground truth. Everything else can be
 correct.

 Label as:
 - "correct" if the predicted description basically captures
 the
 same high-level relationship described by the ground
 truth,
 even if some minor details are omitted.
 - "partially_correct" if it covers some of the core idea but
 introduces a misunderstanding or omits a crucial aspect
 that
 changes the overall meaning.
 - "incorrect" if it is largely or completely inconsistent
 with
 the ground truth (e.g., describing a different class
 relationship
 or functionality).

 Provide:
 - alignment_label: correct, partially_correct, or incorrect
 - justification: short reason
 - error_type: if not correct (e.g., "major_contradiction")

```

```

IMPORTANT:
- Do NOT generate new content; only evaluate alignment.
- Treat missing details or code specifics as unimportant,
 as long as the main relationship is correct.
""",
"output_format": {
 "alignment_label": "string",
 "justification": "string",
 "error_type": "string"
}
}

system_message = (
 "You are an impartial evaluator. Determine whether the
 predicted relationship "
 "is correct, partially_correct, or incorrect relative to the
 ground_truth. "
 "Do not generate a new explanation. Ensure your response is
 in JSON only."
)

```

### A.1 Established Projects Dataset Statistics

Table 16 presents detailed statistics for the 27 Established Projects used in our generalizability evaluation. We report two complexity metrics: *link density* (proportion of true links among all possible pairs) and *combinatorial complexity* ( $C_{avg} = \log_2 \binom{M}{|k|}$  bits), which approximates the difficulty of identifying the correct subset of linked artifacts.

Table 16 summarizes per-project statistics for the Established Projects dataset. Most CoDocBench projects have low combinatorial complexity (4.5–8.2 bits) because each docstring links to exactly one function ( $\bar{k} = 1$ ), while their link densities range from 0.33% to 4.55%. In contrast, the two TransArc systems exhibit denser, multi-link traces: MediaStore averages 2 file-level links per sentence, and TeaStore combines the highest link density (21.2%) with the highest combinatorial complexity (93.2 bits).

### A.2 Candidate-List Position Analysis

Table 17 reports recall by candidate-list position for the unseen datasets and Established Projects. Candidate artifacts were divided into early, middle, and late thirds using normalized position in each project-level candidate artifact list.

Table 16. Statistics of the Established Projects Dataset (25 CoDocBench projects + 2 TransArc systems)

Project	Docs	Artifacts	Traces	$\bar{k}$	Density	$C_{avg}$
AzurLaneAutoScript	123	123	123	1.0	0.81%	6.9
StratosphereLinuxIPS	48	48	48	1.0	2.08%	5.6
TTS	23	23	23	1.0	4.35%	4.5
akshare	37	37	37	1.0	2.70%	5.2
celery	23	23	23	1.0	4.35%	4.5
certbot	27	27	27	1.0	3.70%	4.8
cupy	47	47	47	1.0	2.13%	5.6
diffusers	30	30	30	1.0	3.33%	4.9
espnet	108	108	108	1.0	0.93%	6.8
frappe	38	38	38	1.0	2.63%	5.2
ipython	74	74	74	1.0	1.35%	6.2
manim	29	29	29	1.0	3.45%	4.9
matplotlib	300	300	300	1.0	0.33%	8.2
mediastore	25	15	50	2.0	13.33%	6.7
mindsdb	29	29	29	1.0	3.45%	4.9
pandas	301	301	301	1.0	0.33%	8.2
pyro	34	34	34	1.0	2.94%	5.1
pytorch	151	151	151	1.0	0.66%	7.2
pytorch_geometric	32	32	32	1.0	3.13%	5.0
rasa	107	107	107	1.0	0.93%	6.7
sentry	22	22	22	1.0	4.55%	4.5
teastore	23	145	707	30.7	21.20%	93.2
textual	28	28	28	1.0	3.57%	4.8
tornado	49	49	49	1.0	2.04%	5.6
transformers	64	64	64	1.0	1.56%	6.0
zipline	45	45	45	1.0	2.22%	5.5
zulip	51	51	51	1.0	1.96%	5.7
<b>Total</b>	<b>1,868</b>	<b>1,980</b>	<b>2,577</b>	—	—	—

Table 17. Recall by candidate-list position.

Dataset / Group	Model	Early Recall (%)	Middle Recall (%)	Late Recall (%)
<i>Unseen Projects</i>				
Crawl4AI	Claude 3.5	61.5	91.0	80.7
	GPT-4o	48.8	67.5	59.6
	o3-mini	48.3	82.1	85.2
Unity Catalog	Claude 3.5	78.0	67.8	78.5
	GPT-4o	54.1	45.5	59.6
	o3-mini	36.5	47.8	55.8
<i>Average</i>	Claude 3.5	63.1	81.0	80.0
	GPT-4o	49.3	58.0	59.6
	o3-mini	47.1	67.3	75.5
<i>Established Projects</i>				
CoDocBench	Claude 4	97.6	98.4	98.4
	GPT-4o	98.6	98.5	99.1
	o3-mini	96.4	95.0	97.2
TransArc	Claude 4	24.8	22.1	45.4
	GPT-4o	15.0	16.1	31.0
	o3-mini	9.6	5.7	19.8
<i>Average</i>	Claude 4	77.9	74.8	82.6
	GPT-4o	75.9	73.0	78.8
	o3-mini	72.8	67.4	74.1

*Note:* Early, middle, and late denote normalized thirds of the project-level candidate artifact list. Claude 3.5 was used for the unseen datasets; Claude 4 was used for the Established Projects experiments.